UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Proyecto *Red Social UPT*

Curso: *Patrones de Software*

Docente: *Mag. Ing. Patrick Cuadros Quiroga}*

Integrantes:

Cutipa Gutierrez, Ricardo (2021069827)
Málaga Espinoza, Ivan (2021071086)
Chino Rivera, Angel (2021069830)

Tacna – Perú
2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	10/10/2020	Versión Original

Sistema *Red Social UPT*
Documento de Arquitectura de Software

Versión *1.0*

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	10/10/2020	Versión Original

INDICE GENERAL

Contenido

1. INTRODUCCIÓN 5

1.1. Propósito (Diagrama 4+1) 5

1.2. Alcance 5

1.3. Definición, siglas y abreviaturas 6

1.4. Organización del documento 6

2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS 7

2.1.1. Requerimientos Funcionales 8

2.1.2. Requerimientos No Funcionales – Atributos de Calidad 8

3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA 10

3.1. Vista de Caso de uso 10

3.1.1. Diagramas de Casos de uso 11

3.2. Vista Lógica 14

3.2.1. Diagrama de Subsistemas (paquetes) 14

3.2.2. Diagrama de Secuencia (vista de diseño) 14

3.2.3. Diagrama de Colaboración (vista de diseño) 29

3.2.4. Diagrama de Objetos 30

3.2.5. Diagrama de Clases 30

3.2.6. Diagrama de Base de datos (relacional o no relacional) 31

3.3. Vista de Implementación (vista de desarrollo) 32

3.3.1. Diagrama de arquitectura software (paquetes) 32

3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes) 32

3.4. Vista de procesos 32

3.4.1. Diagrama de Procesos del sistema (diagrama de actividad) 33

3.5. Vista de Despliegue (vista física) 34

3.5.1. Diagrama de despliegue 34

4. ATRIBUTOS DE CALIDAD DEL SOFTWARE 34

Escenario de Funcionalidad 34

Escenario de Usabilidad 35

Escenario de confiabilidad 35

Escenario de rendimiento 36

Escenario de mantenibilidad 36

Otros Escenarios 36

1. INTRODUCCIÓN

1.1. Propósito (Diagrama 4+1)

El presente documento tiene como propósito describir la arquitectura del sistema **Red Social UPT**, proporcionando una visión global y estructurada del diseño basado en el modelo de vistas **4+1**. Este modelo permite representar la arquitectura desde diferentes perspectivas: lógica, de desarrollo, de procesos, física y de escenarios, facilitando la comprensión tanto para desarrolladores como para stakeholders.

La arquitectura del sistema está basada en el enfoque de **microservicios**, implementado mediante el framework PHP/Lumen, lo que permite una alta escalabilidad, mantenibilidad e independencia entre módulos. Se han considerado los requisitos funcionales definidos en el SRS, como la gestión de usuarios, publicaciones, historias y perfiles, así como los requisitos no funcionales, tales como seguridad, rendimiento y disponibilidad.

Dentro de las decisiones arquitectónicas más relevantes, se prioriza la **escalabilidad y seguridad** sobre la portabilidad, dado que el sistema será desplegado en un entorno controlado (VPS Debian con Docker). Asimismo, se garantiza la autenticación segura mediante validación de correos institucionales, lo que influye directamente en el diseño del sistema.

1.2. Alcance

Este documento se centra principalmente en el desarrollo de la **vista lógica** de la arquitectura del sistema Red Social UPT, describiendo los principales componentes, sus responsabilidades y las relaciones entre ellos.

Se incluyen también aspectos generales de otras vistas del modelo 4+1, como la vista de desarrollo (organización en microservicios), la vista física (despliegue en contenedores Docker) y la vista de escenarios (casos de uso principales). Sin embargo, se omiten detalles extensivos de la vista de procesos, debido a que no representa un enfoque crítico en la arquitectura actual del sistema.

El documento cubre los tres microservicios principales:

- Auth Service
- Posts Service
- Profile & Social Service

Además, se consideran los mecanismos de comunicación entre servicios, autenticación, almacenamiento de datos y despliegue en infraestructura definida.

1.3. Definición, siglas y abreviaturas

A continuación, se presentan los términos más relevantes utilizados en el documento:

- **API (Application Programming Interface):** Conjunto de reglas que permite la comunicación entre sistemas.
- **Docker:** Plataforma de contenedores para desplegar aplicaciones de forma aislada.
- **Docker Compose:** Herramienta para definir y ejecutar aplicaciones multicontenedor.
- **Framework:** Estructura base que facilita el desarrollo de software.
- **Lumen:** Micro-framework de PHP utilizado para construir microservicios ligeros.
- **Microservicios:** Arquitectura basada en servicios independientes que se comunican entre sí.
- **SAD (Software Architecture Document):** Documento que describe la arquitectura del sistema.
- **SRS (Software Requirements Specification):** Documento que define los requisitos del sistema.
- **UPT:** Universidad Privada de Tacna.
- **VPS (Virtual Private Server):** Servidor virtual utilizado para desplegar aplicaciones.
- **Terraform:** Herramienta de infraestructura como código para automatizar despliegues.
- **Autenticación:** Proceso de verificación de identidad de un usuario.
- **Feed:** Flujo de publicaciones mostradas al usuario.
- **Historia efímera:** Publicación temporal que desaparece después de un tiempo determinado.

1.4. Organización del documento

El presente documento de arquitectura de software (SAD) se encuentra organizado de la siguiente manera:

Sección 1: Introducción
Presenta el propósito del documento, el alcance, las definiciones, siglas y abreviaturas utilizadas, así como la estructura general del documento.

Sección 2: Objetivos y Restricciones Arquitectónicas
Describe los objetivos principales del sistema desde el punto de vista arquitectónico, considerando los requerimientos funcionales y no funcionales. Asimismo, se detallan las restricciones que influyen en el diseño, como tecnologías, infraestructura y decisiones de implementación.

Sección 3: Representación de la Arquitectura del Sistema
Expone la arquitectura del sistema utilizando diferentes vistas del modelo 4+1:

- **Vista de Casos de Uso:** Describe los escenarios principales de interacción entre los usuarios y el sistema, incluyendo los diagramas de casos de uso.
- **Vista Lógica:** Define la estructura interna del sistema mediante diagramas de subsistemas, clases, objetos, secuencia, colaboración y base de datos.

- **Vista de Implementación (Desarrollo):** Muestra la organización del sistema en términos de componentes y paquetes de software.
- **Vista de Procesos:** Describe el comportamiento dinámico del sistema a través de diagramas de actividades.
- **Vista de Despliegue (Física):** Representa la distribución del sistema en la infraestructura, incluyendo el uso de contenedores y servidores.

Sección 4: Atributos de Calidad del Software
Define los principales atributos de calidad del sistema mediante escenarios específicos, tales como funcionalidad, usabilidad, confiabilidad, rendimiento y mantenibilidad, así como otros escenarios relevantes para evaluar el comportamiento del sistema.

2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS

2.1. Priorización de requerimientos

A continuación, se presenta la priorización general de los requerimientos del sistema, la cual define el orden de implementación y la importancia dentro de la arquitectura.

ID	Descripción	Prioridad
RF-01	Autenticación institucional con Google OAuth	Alta
RF-02	Creación de publicaciones con control de visibilidad	Alta
RF-03	Feed cronológico filtrado	Alta
RF-04	Sistema de likes	Media
RF-05	Comentarios y likes en comentarios	Media
RF-06	Gestión de perfil	Media
RF-07	Directorio de compañeros	Media
RF-08	Chat privado	Baja
RF-09	Panel de administración	Alta
RNF-01	Acceso exclusivo con dominio institucional	Alta
RNF-02	Autenticación delegada a Google	Alta
RNF-03	Comunicación segura con JWT	Alta
RNF-04	Calidad de código (SonarCloud)	Media
RNF-05	Seguridad de dependencias (Snyk)	Media
RNF-06	CI/CD con GitHub Actions	Media

ID	Descripción	Prioridad
RNF-07	Despliegue en VPS con Docker	Alta
RNF-08	Infraestructura con Terraform	Media
RNF-09	Documentación (README)	Baja
RNF-10	Wiki del proyecto	Baja

2.1.1. Requerimientos Funcionales

ID	Descripción	Prioridad
RF-01	Autenticación institucional con Google OAuth y validación @virtual.upt.pe	Alta
RF-02	Creación de publicaciones con texto, imágenes y control de visibilidad	Alta
RF-03	Feed cronológico con filtrado por relaciones y visibilidad	Alta
RF-04	Sistema de likes en publicaciones	Media
RF-05	Comentarios en publicaciones con likes en comentarios	Media
RF-06	Gestión de perfil de usuario (bio, fotos, datos académicos)	Media
RF-07	Directorio de compañeros con solicitudes	Media
RF-08	Chat privado entre compañeros	Baja
RF-09	Panel de administración (usuarios y moderación)	Alta

2.1.2. Requerimientos No Funcionales – Atributos de Calidad

ID	Descripción	Prioridad
RNF-01	Acceso exclusivo mediante dominio @virtual.upt.pe	Alta
RNF-02	Autenticación delegada a Google (sin contraseñas propias)	Alta
RNF-03	Comunicación segura entre microservicios mediante JWT	Alta
RNF-04	Código sin vulnerabilidades críticas (SonarCloud)	Media
RNF-05	Dependencias sin vulnerabilidades críticas (Snyk)	Media
RNF-06	Pipeline CI/CD automatizado	Media

ID	Descripción	Prioridad
RNF-07	Despliegue en VPS Debian con Docker	Alta
RNF-08	Infraestructura como código con Terraform	Media
RNF-09	Documentación completa del proyecto	Baja
RNF-10	Wiki con roadmap del sistema	Baja

Los **atributos de calidad (QAs)** representan propiedades medibles del sistema, tales como seguridad, rendimiento, mantenibilidad y disponibilidad. Estos atributos son fundamentales en la arquitectura, ya que el sistema puede cumplir con su funcionalidad, pero fallar si no satisface estos criterios de calidad.

En este proyecto, los atributos más relevantes son:

- **Seguridad:** control de acceso mediante OAuth y JWT
- **Escalabilidad:** uso de microservicios independientes
- **Rendimiento:** carga eficiente del feed y polling en chat
- **Mantenibilidad:** separación por servicios y uso de Docker
- **Disponibilidad:** despliegue en VPS con contenedores

2.2. Restricciones

Las siguientes restricciones condicionan el diseño e implementación del sistema:

ID	Descripción
RE-01	Solo usuarios con correo @virtual.upt.pe pueden autenticarse
RE-02	El sistema requiere conexión a Internet
RE-03	El almacenamiento de imágenes está limitado al espacio de la VPS
RE-04	Uso obligatorio de arquitectura de microservicios
RE-05	Cada microservicio debe tener su propia base de datos
RE-06	Uso de Docker y Docker Compose para despliegue
RE-07	Infraestructura gestionada mediante Terraform
RE-08	Autenticación obligatoria mediante Google OAuth
RE-09	Comunicación entre servicios mediante JWT
RE-10	Uso de tecnologías específicas: PHP/Lumen, MySQL, JavaScript

3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA

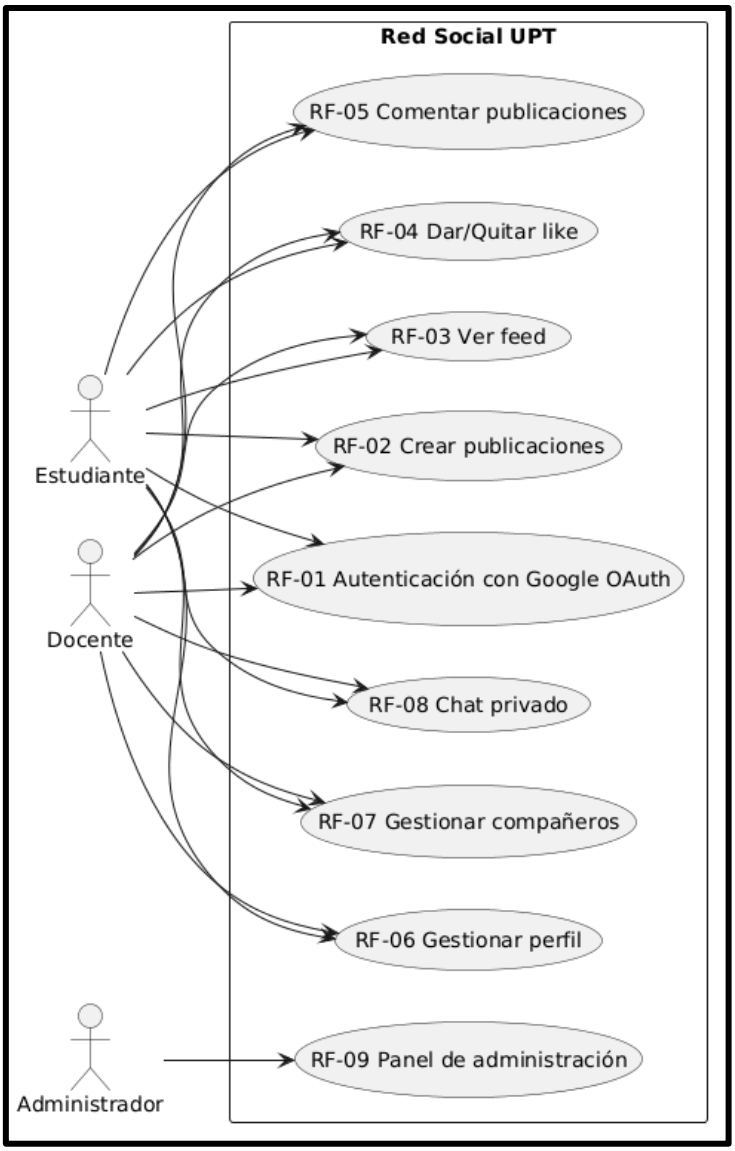
3.1. Vista de Caso de uso

La vista de casos de uso del sistema **Red Social UPT** describe las principales funcionalidades desde la perspectiva del usuario, identificando los actores que interactúan con el sistema y las operaciones que pueden realizar.
Los actores principales del sistema son:

- **Estudiante:** usuario principal que interactúa con la plataforma.
- **Docente:** usuario con funcionalidades similares a estudiante.
- **Administrador:** encargado de la gestión, supervisión y moderación del sistema.

Los casos de uso representan las funcionalidades más importantes del sistema, especialmente aquellas que tienen impacto directo en la arquitectura basada en microservicios.
Los casos de uso críticos identificados son:

- Autenticación mediante Google OAuth
- Visualización del feed
- Creación de publicaciones
- Interacción con publicaciones (likes y comentarios)
- Gestión de perfil
- Administración del sistema



3.1.1. Diagramas de Casos de uso

La descripción de la estructura se ilustra utilizando un conjunto de escenarios de casos de uso, los cuales permiten representar la interacción entre los actores y el sistema.
Estos escenarios describen la secuencia de interacciones entre los diferentes componentes, permitiendo identificar y validar el diseño arquitectónico del sistema.

Diagrama: RF-01 Autenticación con Google OAuth
Este diagrama representa el proceso de autenticación del usuario mediante Google OAuth, validando el dominio institucional antes de permitir el acceso al sistema.

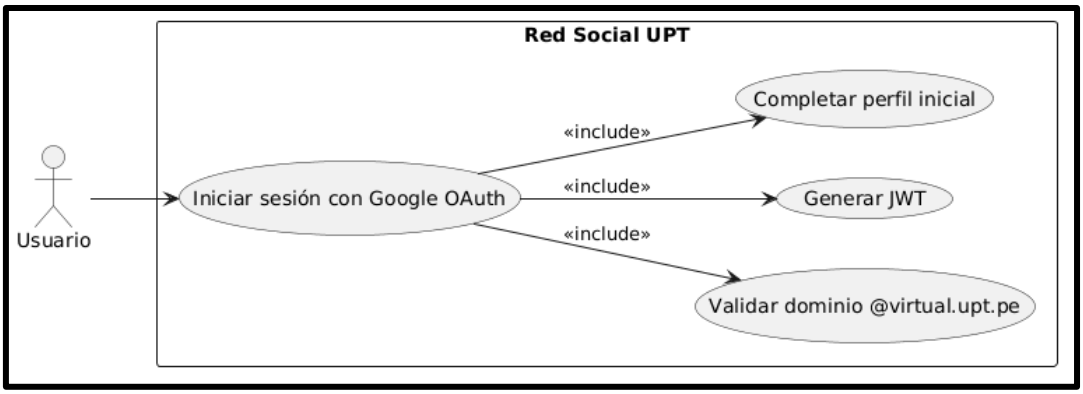


Diagrama: RF-02 Creación de Publicaciones

Este diagrama muestra la interacción del usuario con el sistema para crear publicaciones con contenido multimedia y control de visibilidad.

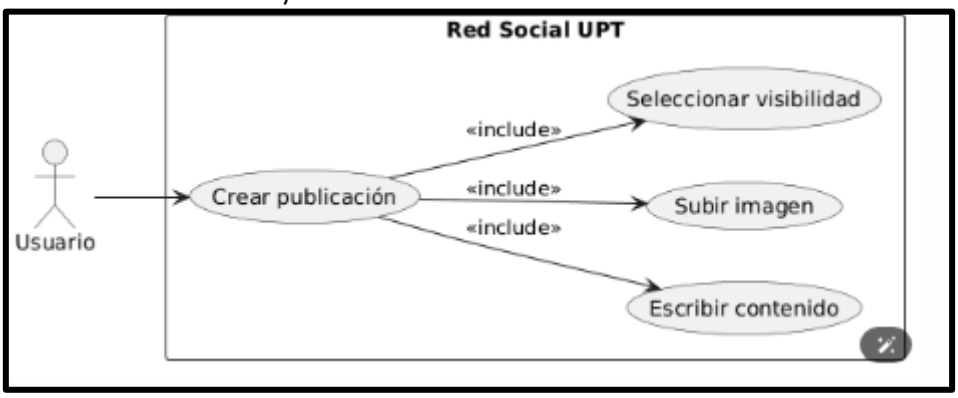


Diagrama: RF-03 Visualización del Feed

Este diagrama representa cómo el usuario accede al feed y visualiza publicaciones filtradas según sus relaciones y permisos.

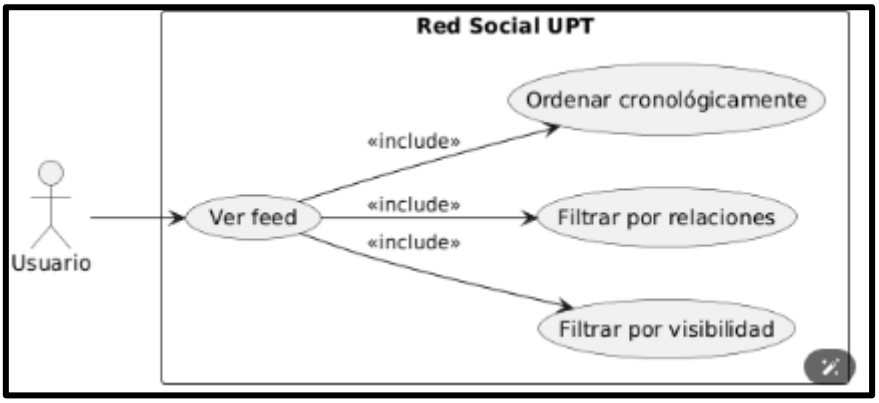


Diagrama: RF-04 Sistema de Likes

Este diagrama describe la interacción del usuario al dar o quitar “me gusta” a una publicación.

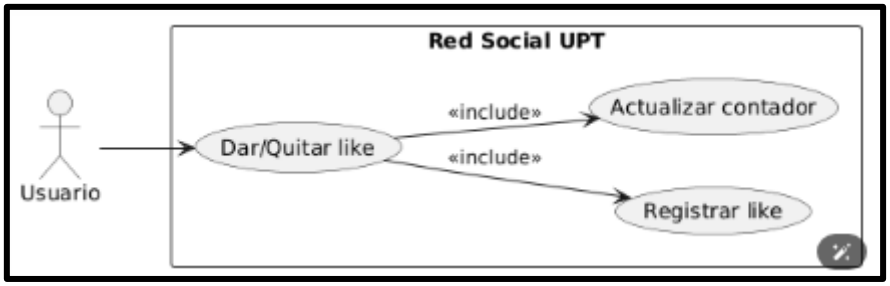


Diagrama: RF-05 Comentarios en Publicaciones
Este diagrama representa el proceso de agregar comentarios y reaccionar a ellos dentro de una publicación.

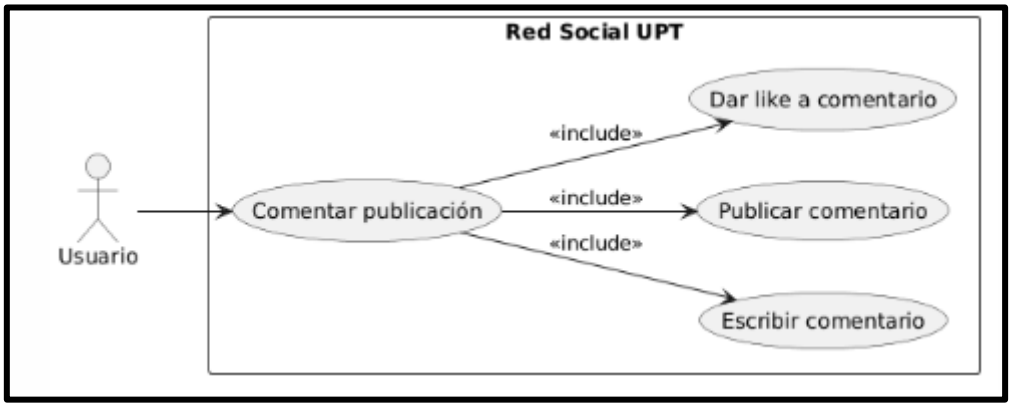


Diagrama: RF-06 Gestión de Perfil
Este diagrama muestra las acciones del usuario para visualizar y editar su perfil personal.

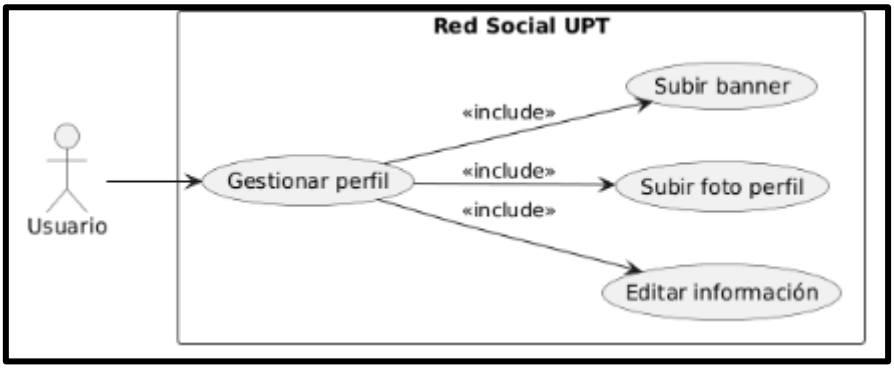


Diagrama: RF-07 Gestión de Compañeros
Este diagrama describe el proceso de envío, aceptación o rechazo de solicitudes de compañeros.

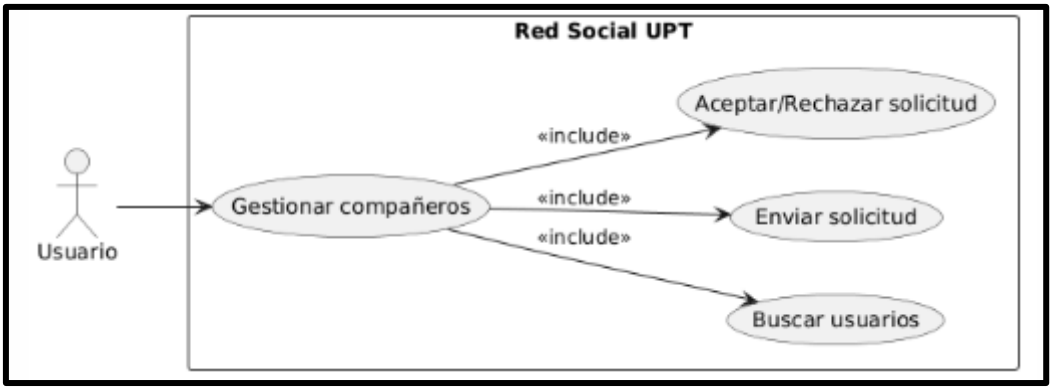


Diagrama: RF-08 Chat Privado
Este diagrama representa la interacción entre usuarios para el envío y recepción de mensajes.

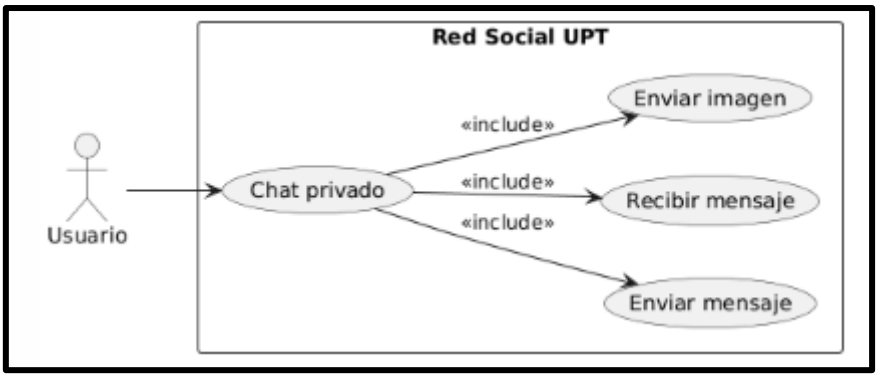
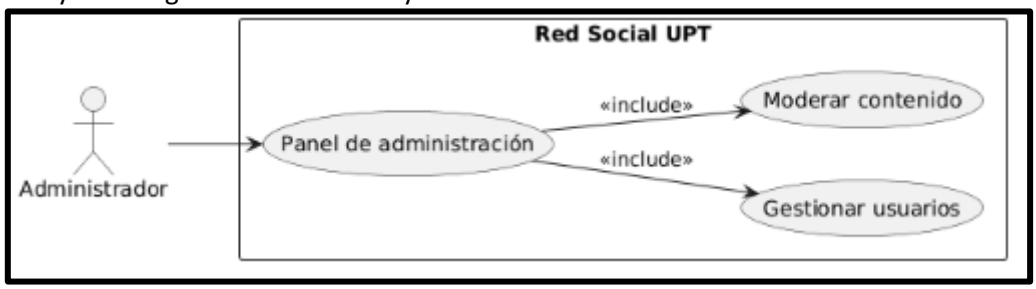
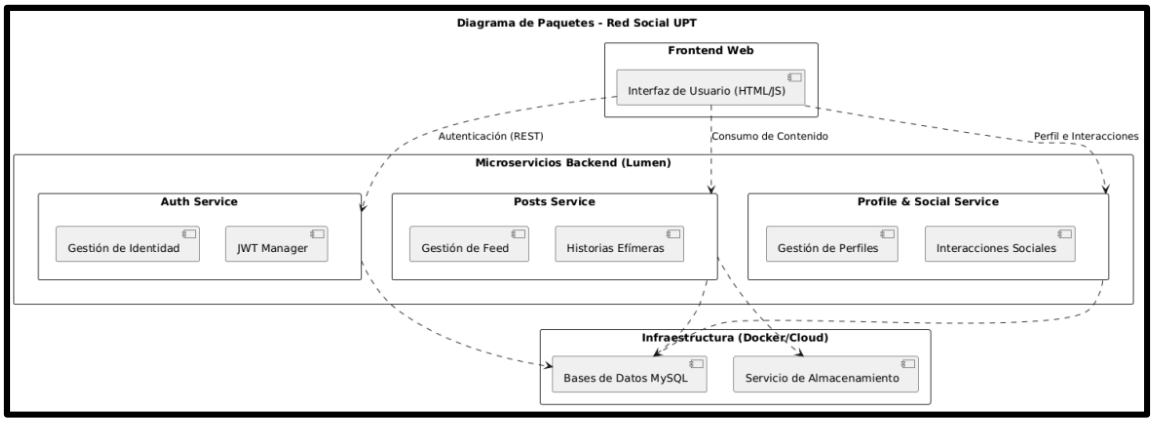


Diagrama: RF-09 Panel de Administración
Este diagrama muestra las funcionalidades disponibles para el administrador, incluyendo la gestión de usuarios y moderación de contenido.



3.2. Vista Lógica

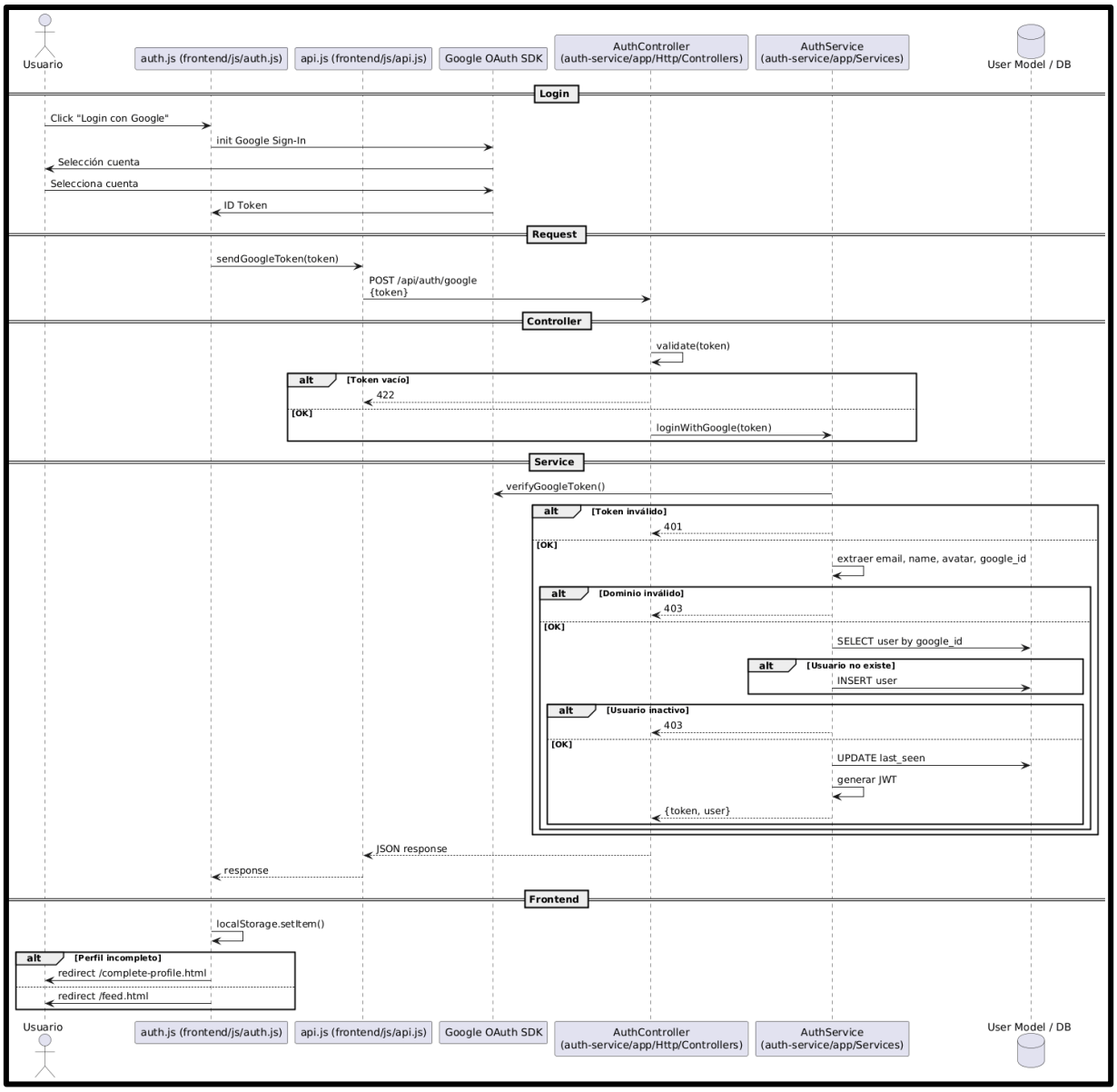
3.2.1. Diagrama de Subsistemas (paquetes)



3.2.2. Diagrama de Secuencia (vista de diseño)

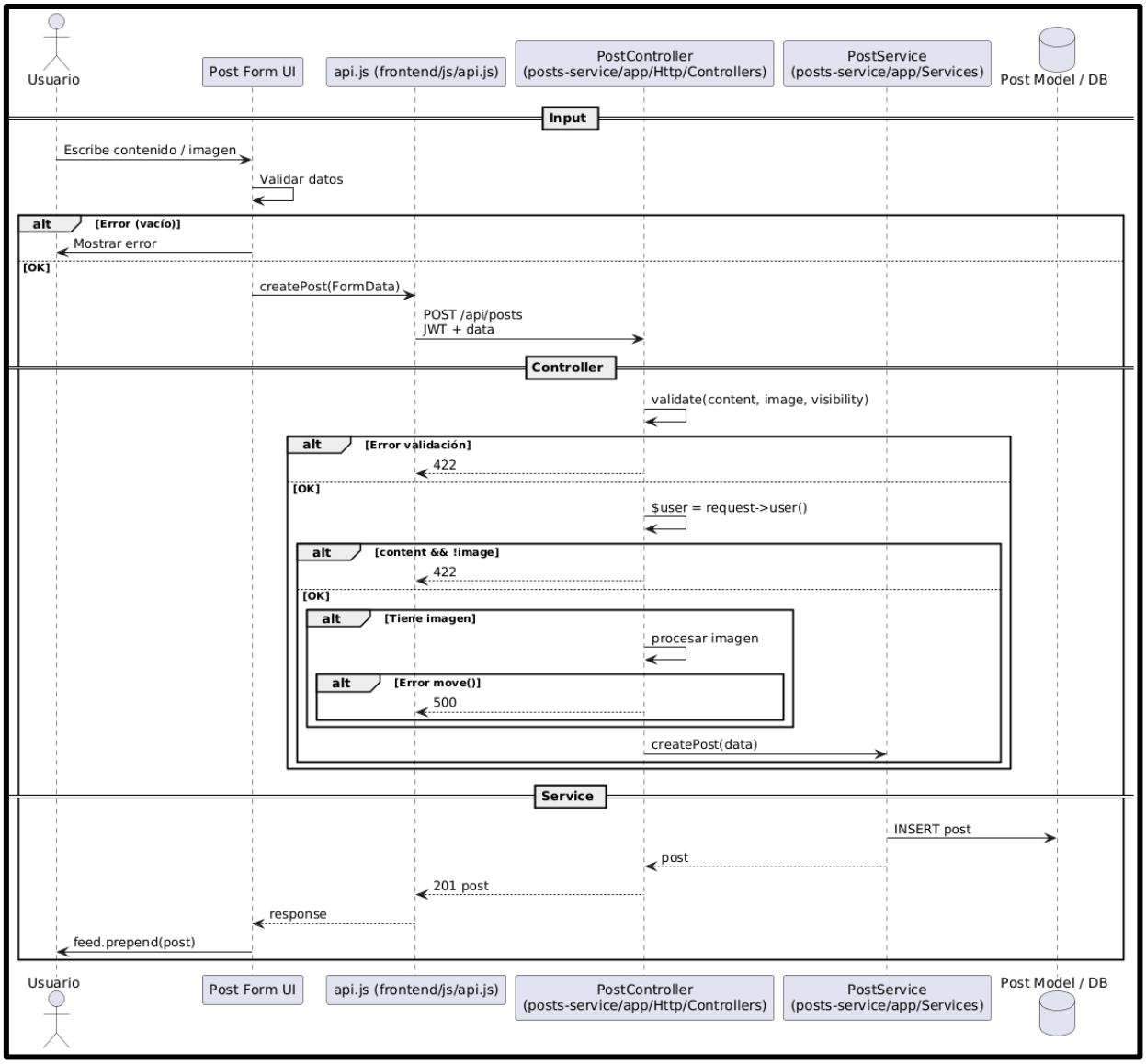
RF-01: Autenticación Institucional con Google OAuth

Se describe el proceso mediante el cual el usuario inicia sesión utilizando su cuenta institucional a través de Google OAuth. El sistema valida el token recibido, verifica el dominio del correo y gestiona la creación o recuperación del usuario en la base de datos, para finalmente generar un token JWT que permite el acceso al sistema.



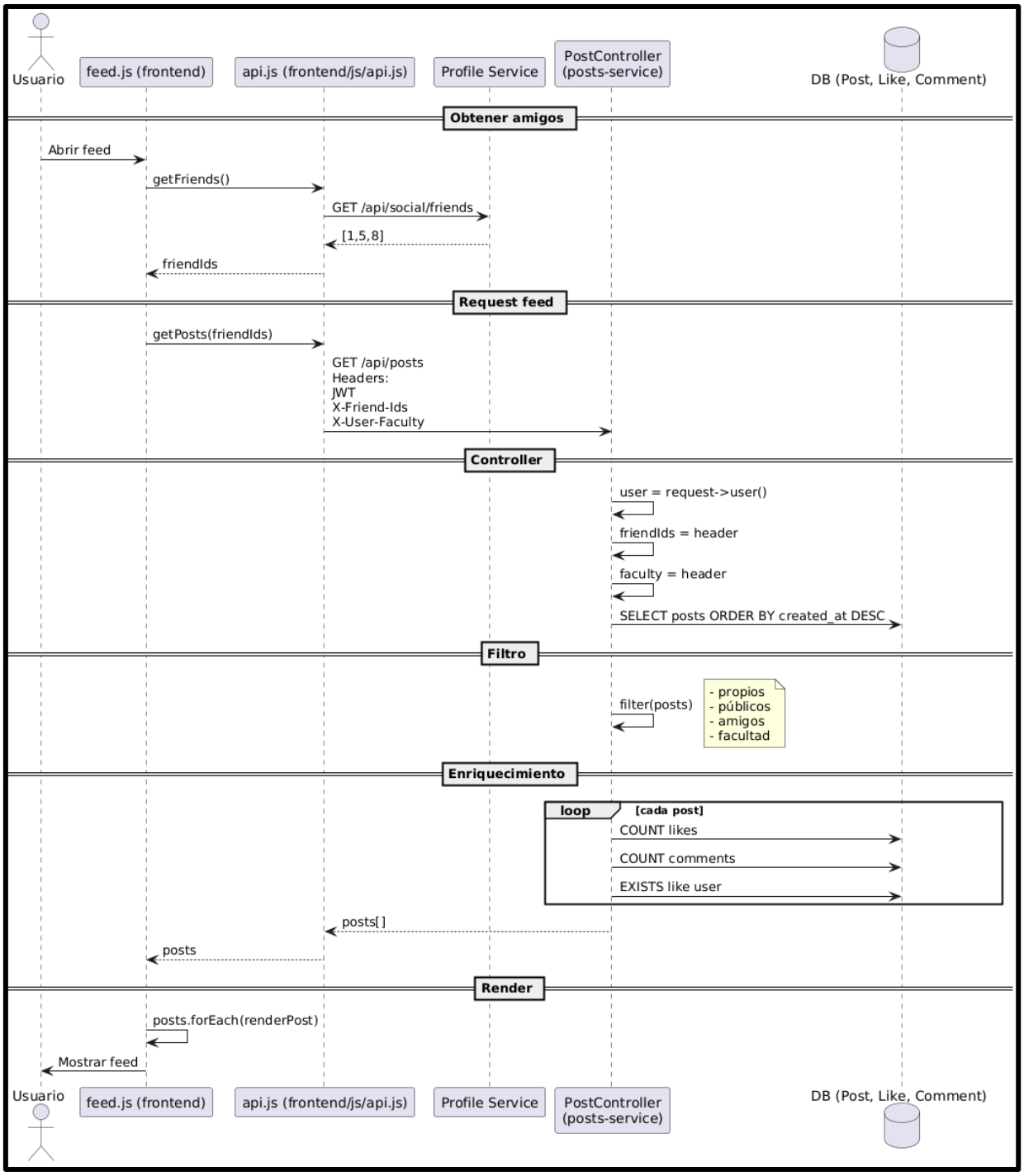
RF-02: Creación de Publicaciones con Control de Visibilidad

Se representa el flujo mediante el cual el usuario crea una publicación con contenido textual y/o imagen, seleccionando el nivel de visibilidad. El sistema valida los datos ingresados, procesa la imagen en caso exista y almacena la publicación junto con los datos del autor en la base de datos.



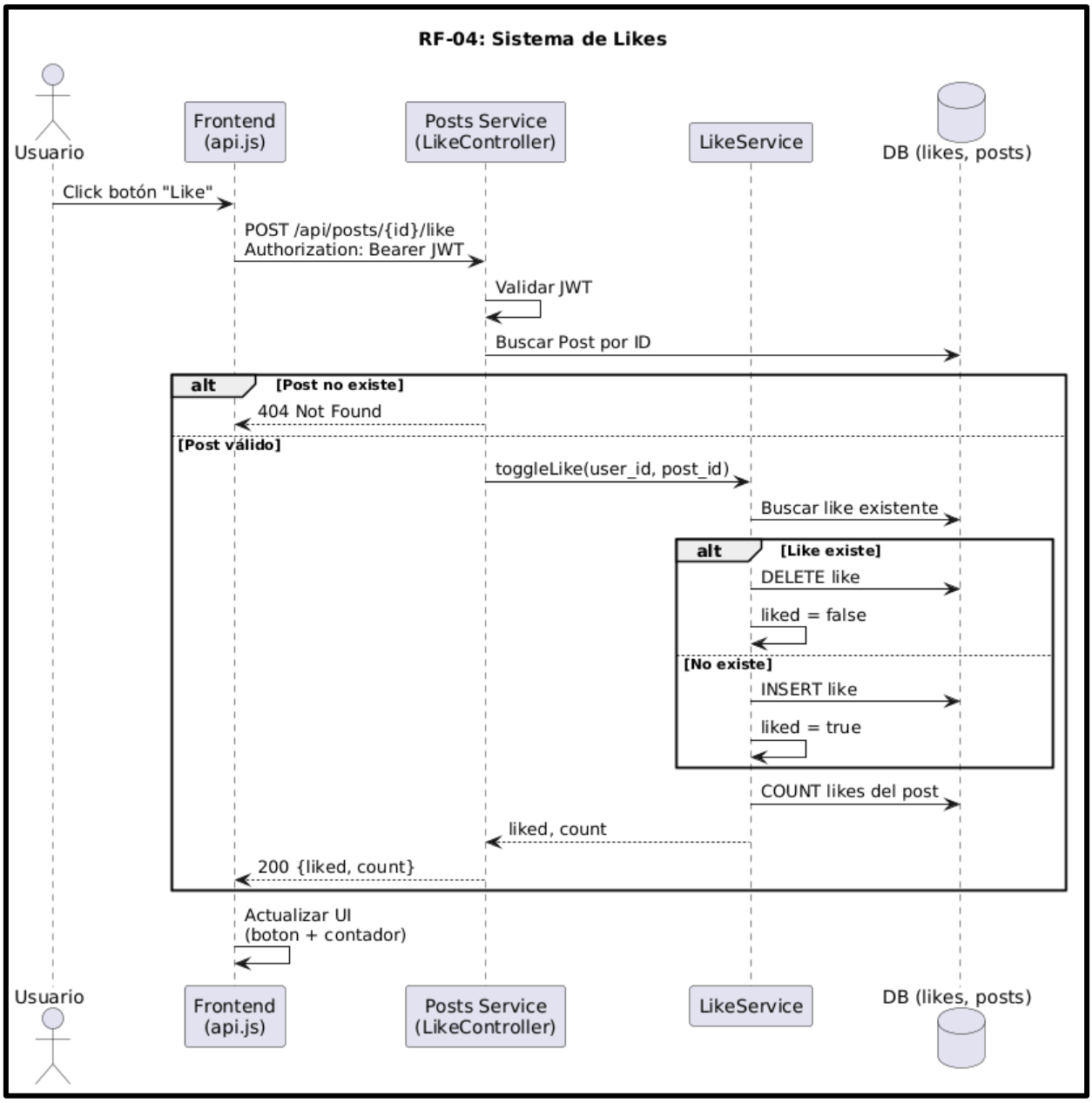
RF-03: Feed Cronológico con Filtrado por Relaciones y Visibilidad

Se muestra el proceso de carga del feed principal, donde el sistema obtiene las publicaciones almacenadas y aplica filtros según la relación entre usuarios y la configuración de visibilidad. Finalmente, se retorna al usuario un listado ordenado cronológicamente.



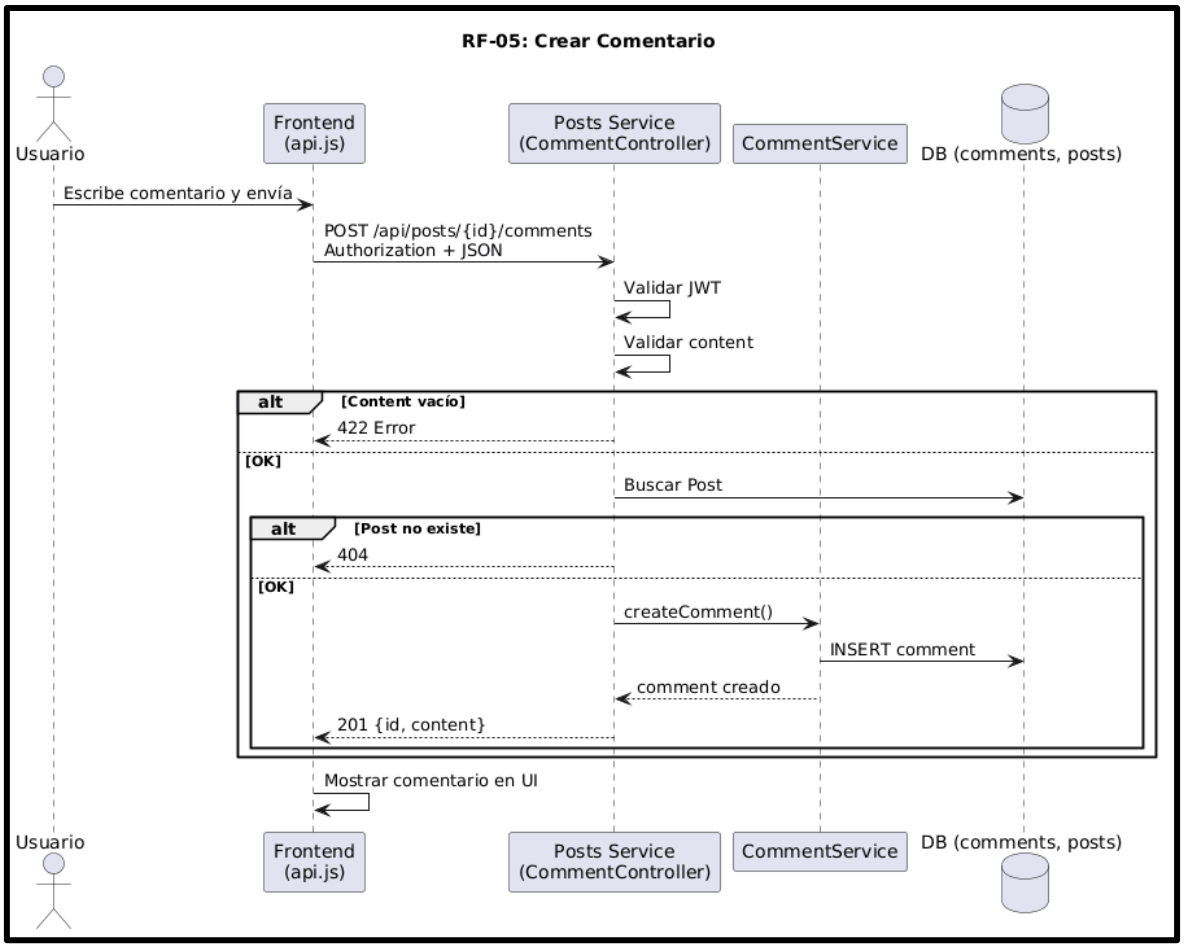
RF-04: Sistema de Likes en Publicaciones

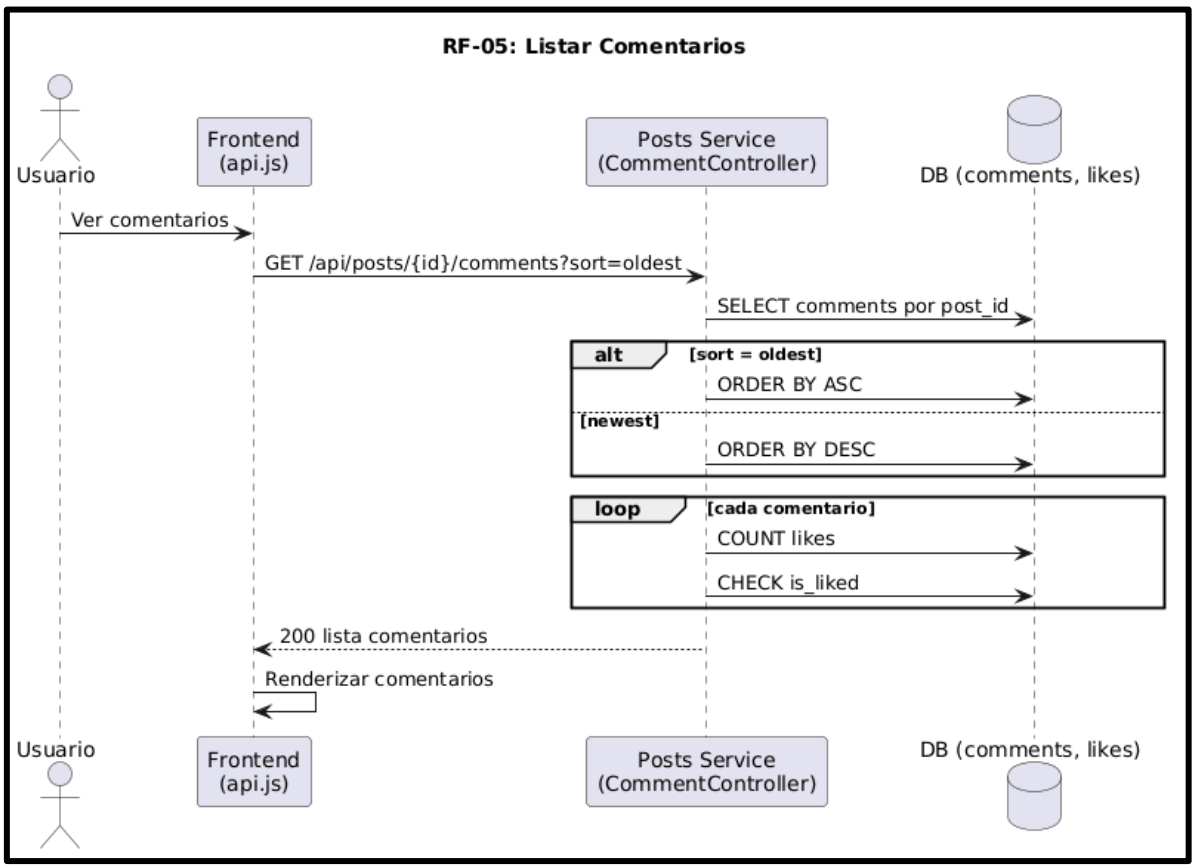
Se describe la interacción del usuario al dar o quitar un “like” en una publicación. El sistema valida la existencia de la publicación, verifica si el usuario ya ha reaccionado y actualiza el estado correspondiente en la base de datos.



RF-05: Comentarios en Publicaciones con Likes en Comentarios

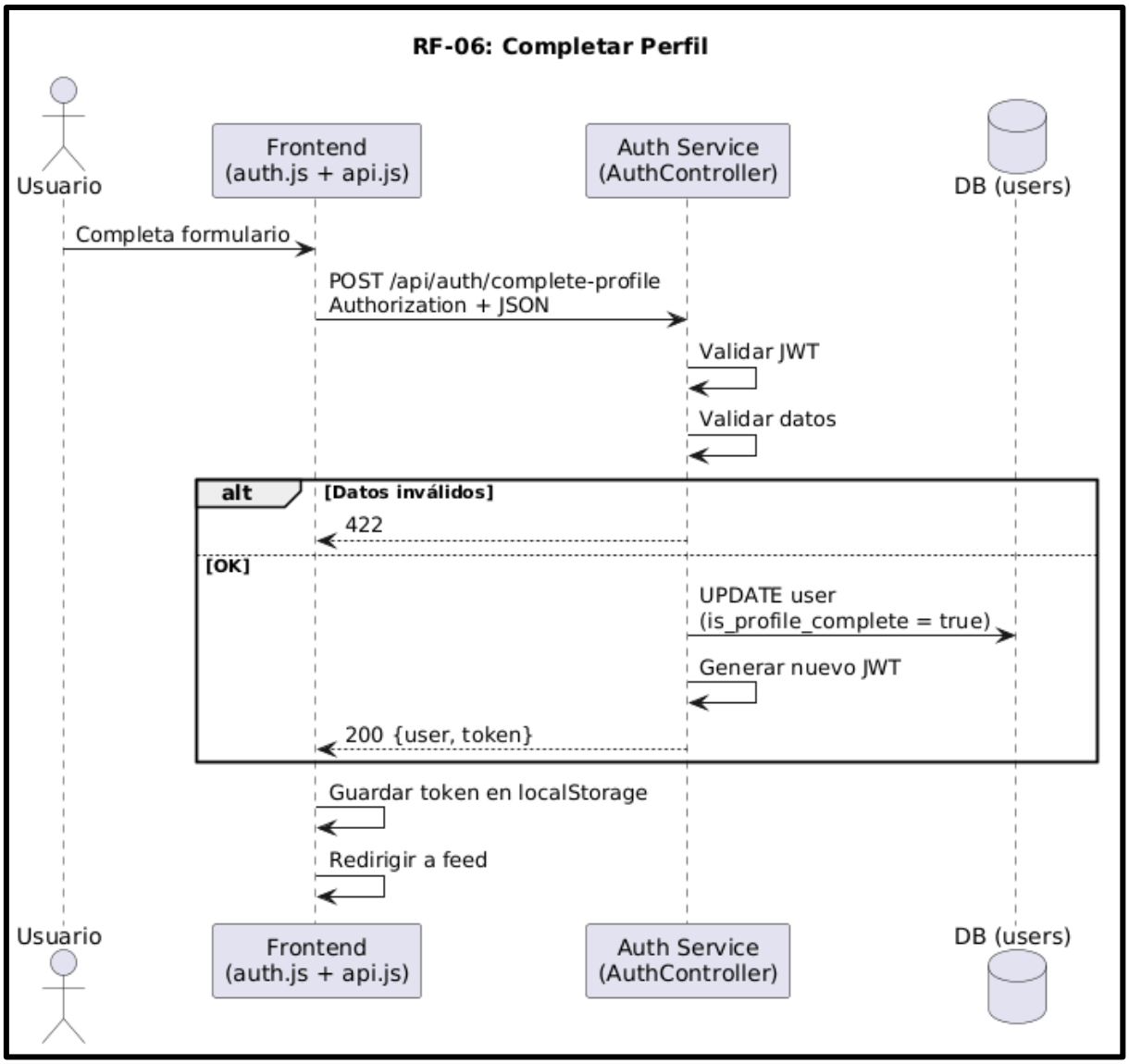
Se representa el flujo de creación y visualización de comentarios en publicaciones. El sistema valida el contenido ingresado, almacena el comentario y permite la interacción mediante likes, actualizando los contadores correspondientes.

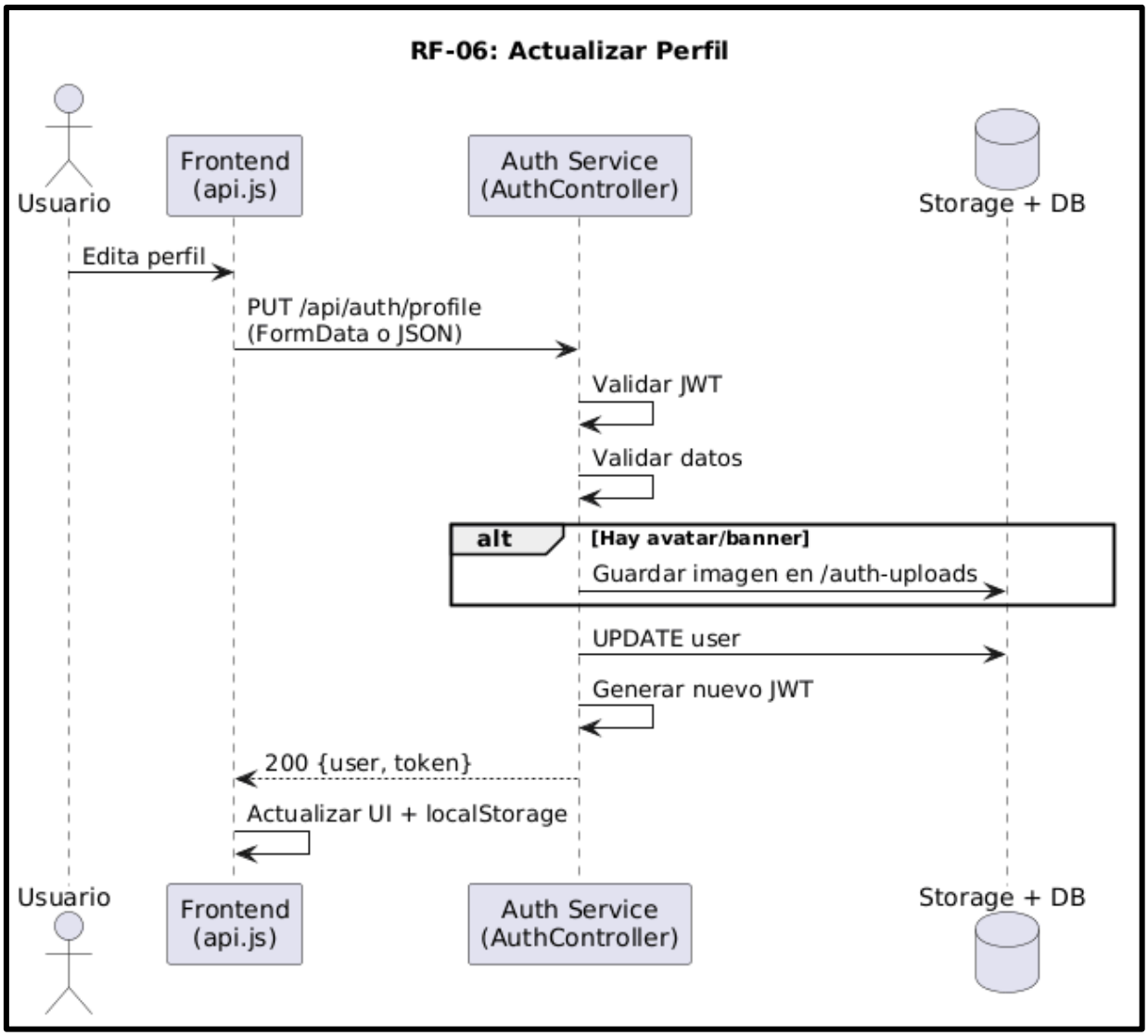

RF-05: Crear Comentario
Usuario
Frontend (api.js)
Posts Service (CommentController)
CommentService
DB (comments, posts)
Escribe comentario y envía
POST /api/posts/{id}/comments
Authorization + JSON
Validar JWT
Validar content
alt
[Content vacío]
422 Error
[OK]
Buscar Post
alt
[Post no existe]
404
[OK]
createComment()
INSERT comment
comment creado
201 {id, content}
Mostrar comentario en UI
Usuario
Frontend (api.js)
Posts Service (CommentController)
CommentService
DB (comments, posts)

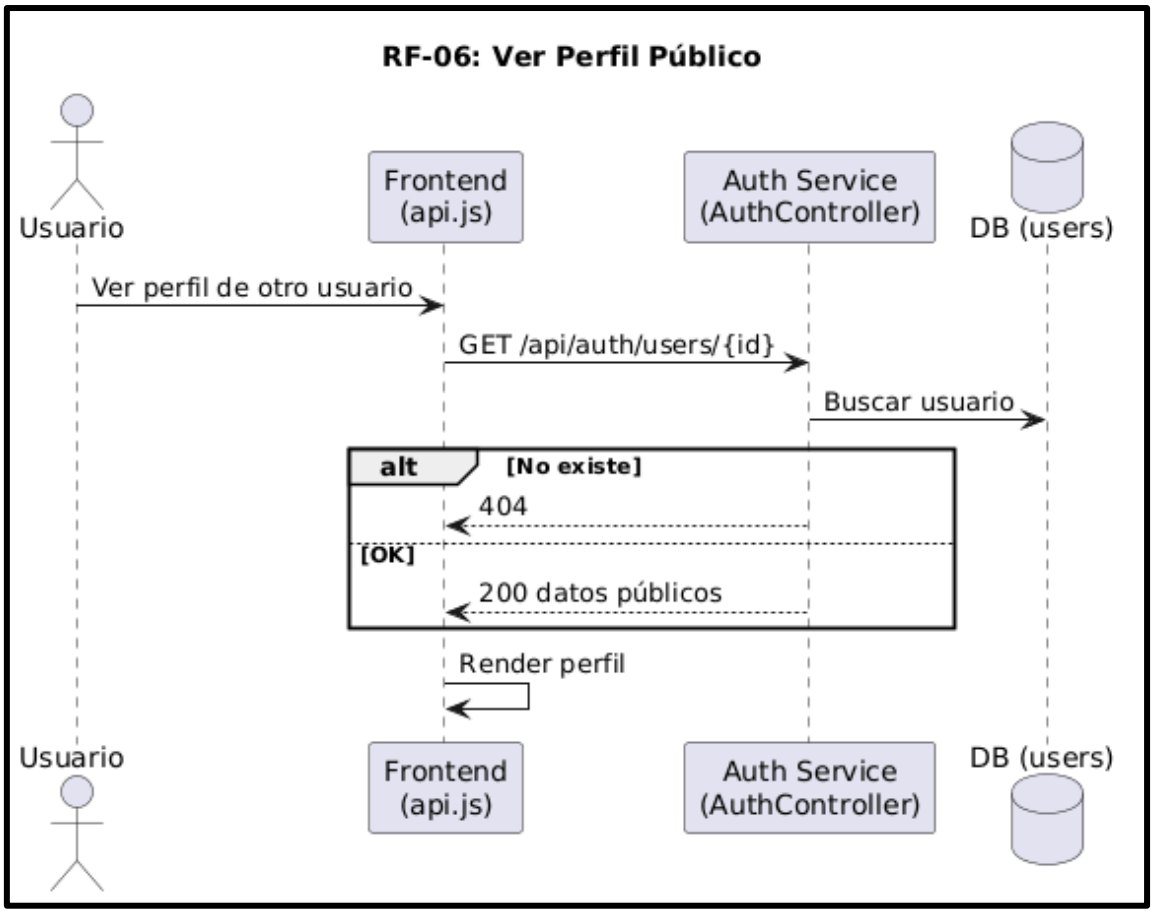

RF-05: Listar Comentarios
Usuario
Frontend (api.js)
Posts Service (CommentController)
DB (comments, likes)
Ver comentarios
GET /api/posts/{id}/comments?sort=oldest
SELECT comments por post_id
alt
[sort = oldest]
ORDER BY ASC
[newest]
ORDER BY DESC
loop
[cada comentario]
COUNT likes
CHECK is_liked
200 lista comentarios
Renderizar comentarios
Usuario
Frontend (api.js)
Posts Service (CommentController)
DB (comments, likes)

RF-06: Gestión de Perfil de Usuario

Se describe el proceso mediante el cual el usuario completa y actualiza su información personal. El sistema valida los datos, gestiona la subida de imágenes (avatar y banner) y actualiza la información en la base de datos, generando un nuevo token con los datos actualizados.

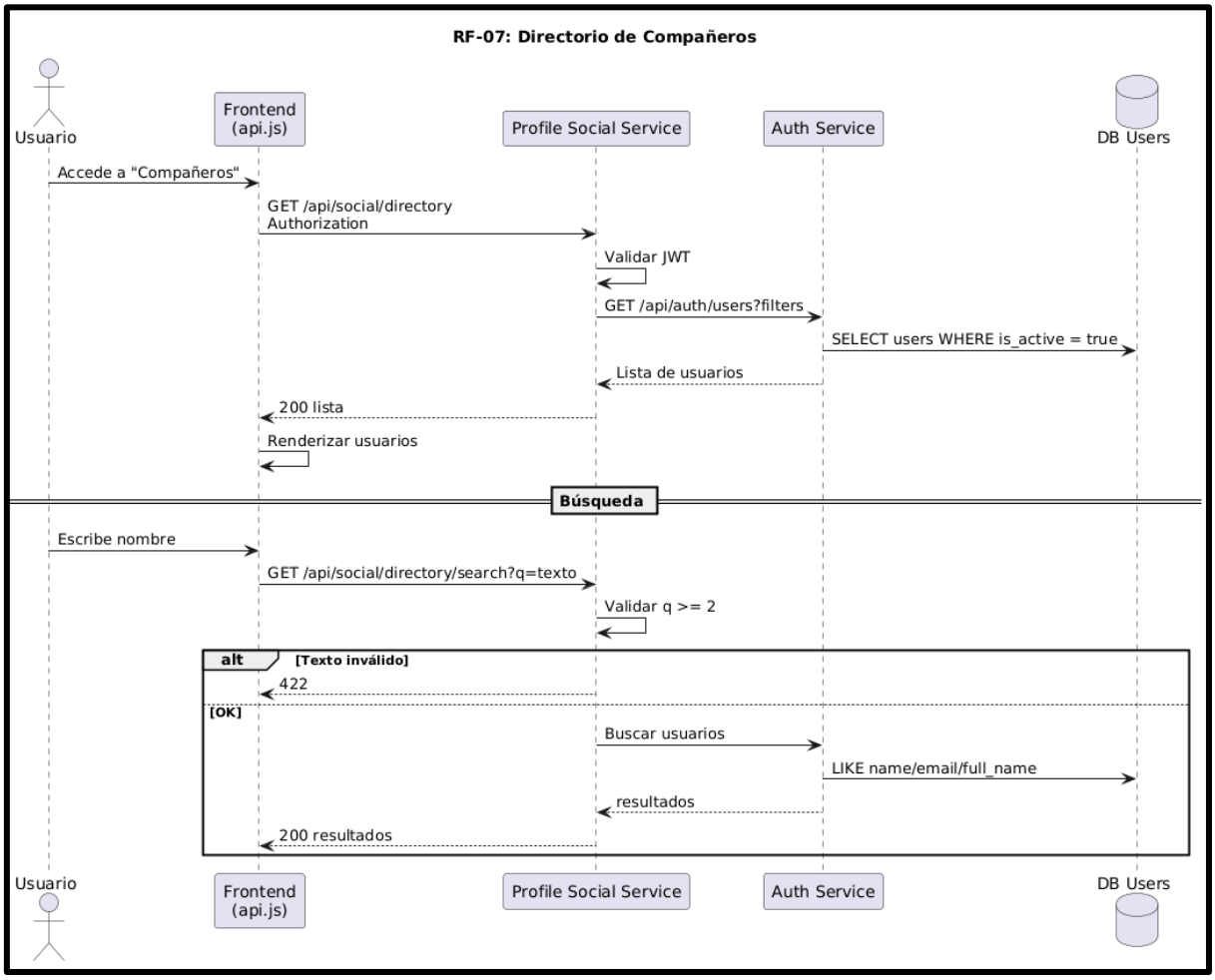


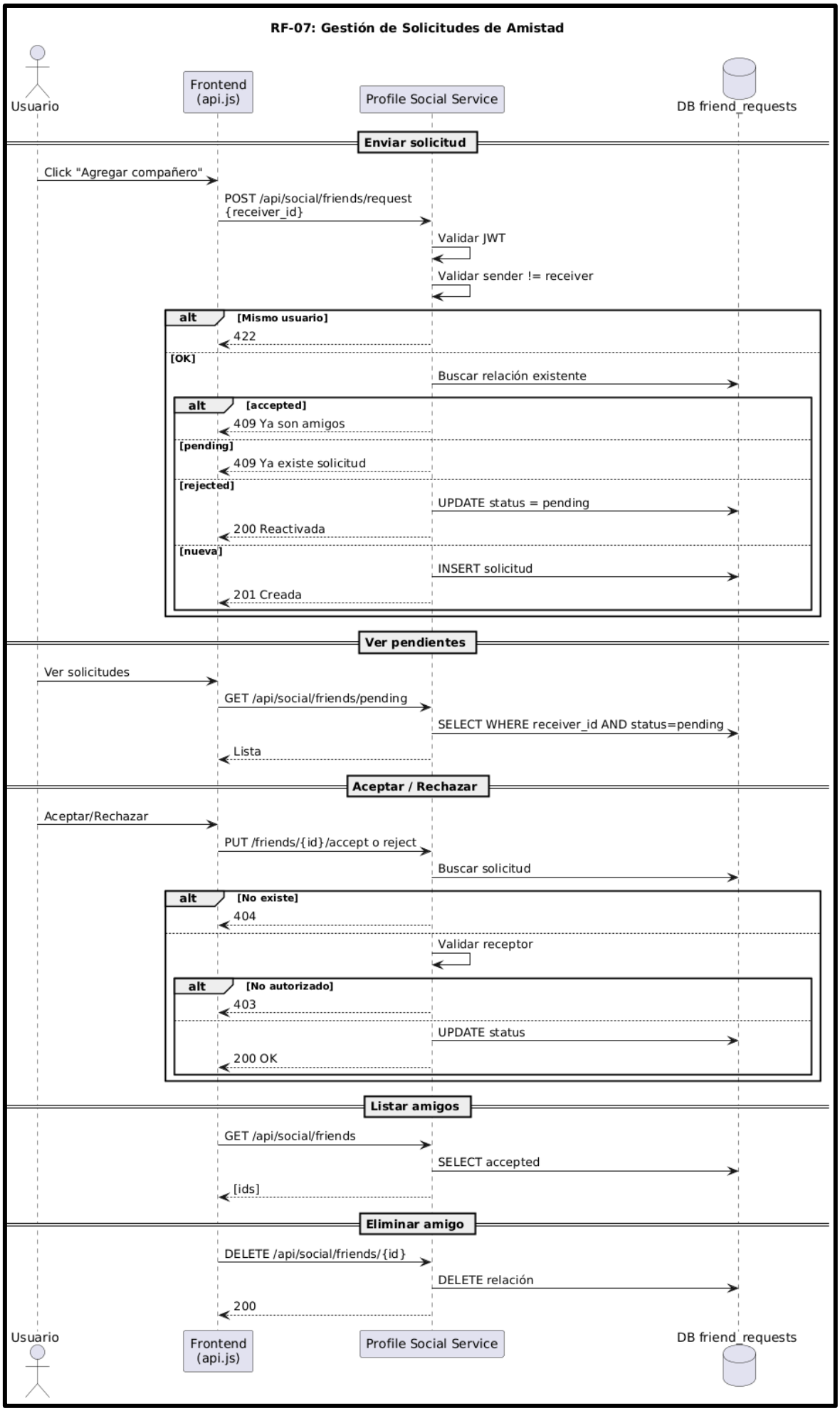

RF-06: Actualizar Perfil
Usuario
Frontend (api.js)
Auth Service (AuthController)
Storage + DB
Edita perfil
PUT /api/auth/profile (FormData o JSON)
Validar JWT
Validar datos
alt
[Hay avatar/banner]
Guardar imagen en /auth-uploads
UPDATE user
Generar nuevo JWT
200 {user, token}
Actualizar UI + localStorage
Usuario
Frontend (api.js)
Auth Service (AuthController)
Storage + DB


RF-06: Ver Perfil Público
Usuario
Frontend (api.js)
Auth Service (AuthController)
DB (users)
Ver perfil de otro usuario
GET /api/auth/users/{id}
Buscar usuario
alt
[No existe]
404
[OK]
200 datos públicos
Render perfil
Usuario
Frontend (api.js)
Auth Service (AuthController)
DB (users)

RF-07: Directorio de Compañeros con Sistema de Solicitudes

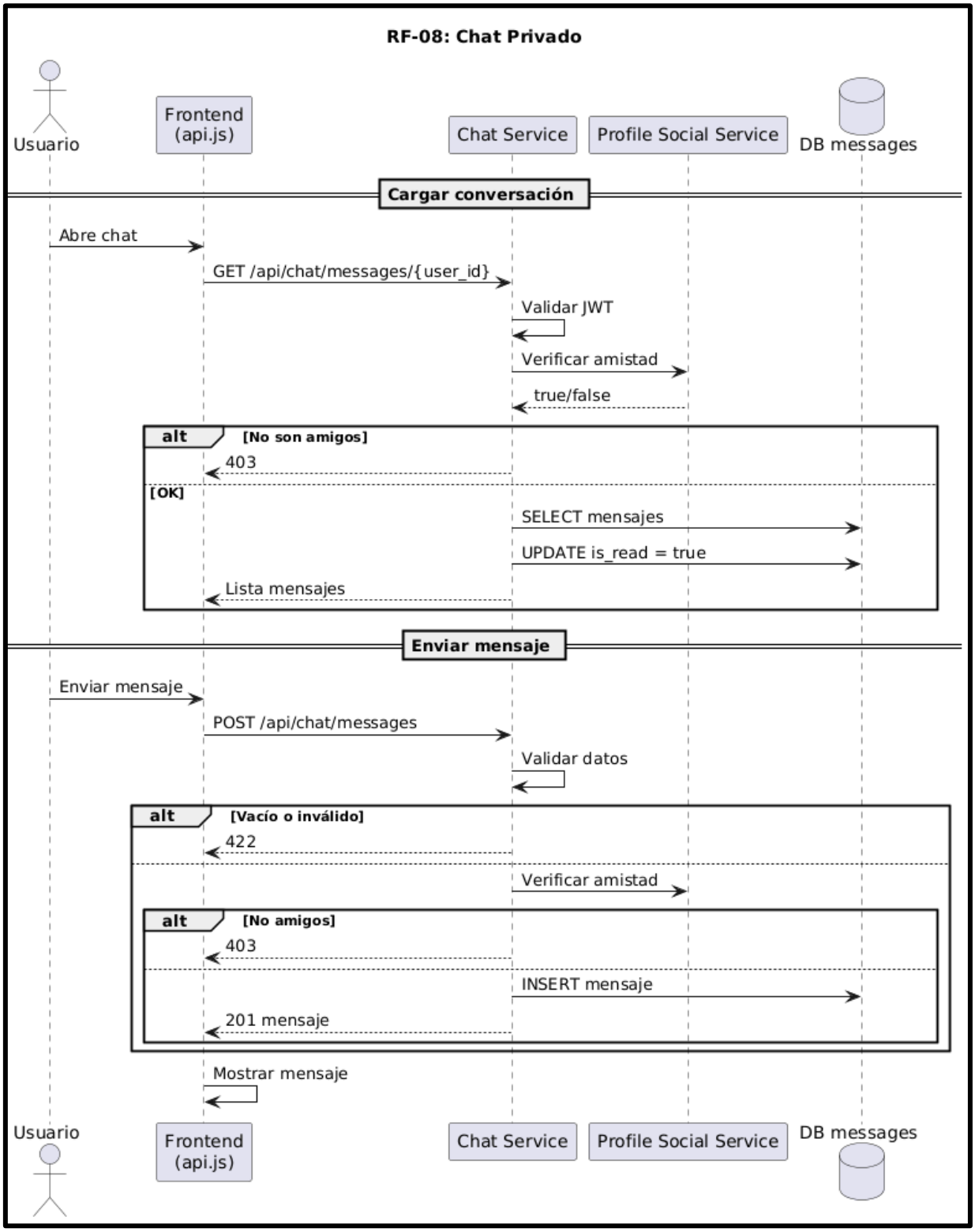
Se representa el flujo de interacción entre usuarios para enviar, aceptar o rechazar solicitudes de amistad. El sistema valida las condiciones de la solicitud y gestiona el estado de la relación entre usuarios.

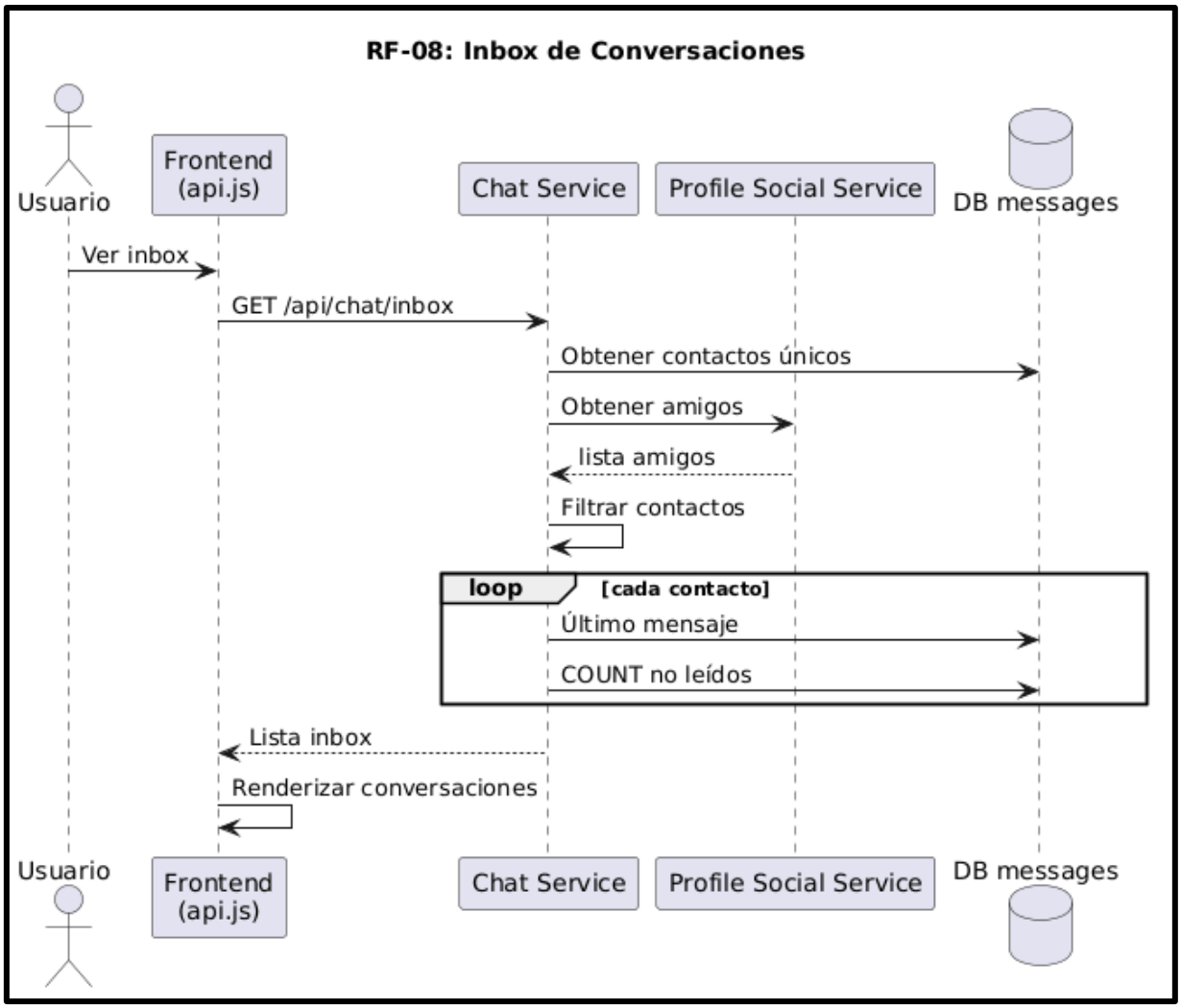



RF-07: Gestión de Solicitudes de Amistad
Usuario
Frontend (api.js)
Profile Social Service
DB friend_requests
Enviar solicitud
Click "Agregar compañero"
POST /api/social/friends/request {receiver_id}
Validar JWT
Validar sender != receiver
alt [Mismo usuario]
422
[OK]
Buscar relación existente
alt [accepted]
409 Ya son amigos
[pending]
409 Ya existe solicitud
[rejected]
UPDATE status = pending
200 Reactivada
[nueva]
INSERT solicitud
201 Creada
Ver pendientes
Ver solicitudes
GET /api/social/friends/pending
SELECT WHERE receiver_id AND status=pending
Lista
Aceptar / Rechazar
Aceptar/Rechazar
PUT /friends/{id}/accept o reject
Buscar solicitud
alt [No existe]
404
Validar receptor
alt [No autorizado]
403
UPDATE status
200 OK
Listar amigos
GET /api/social/friends
SELECT accepted
[ids]
Eliminar amigo
DELETE /api/social/friends/{id}
DELETE relación
200
Usuario
Frontend (api.js)
Profile Social Service
DB friend_requests

RF-08: Chat Privado entre Compañeros

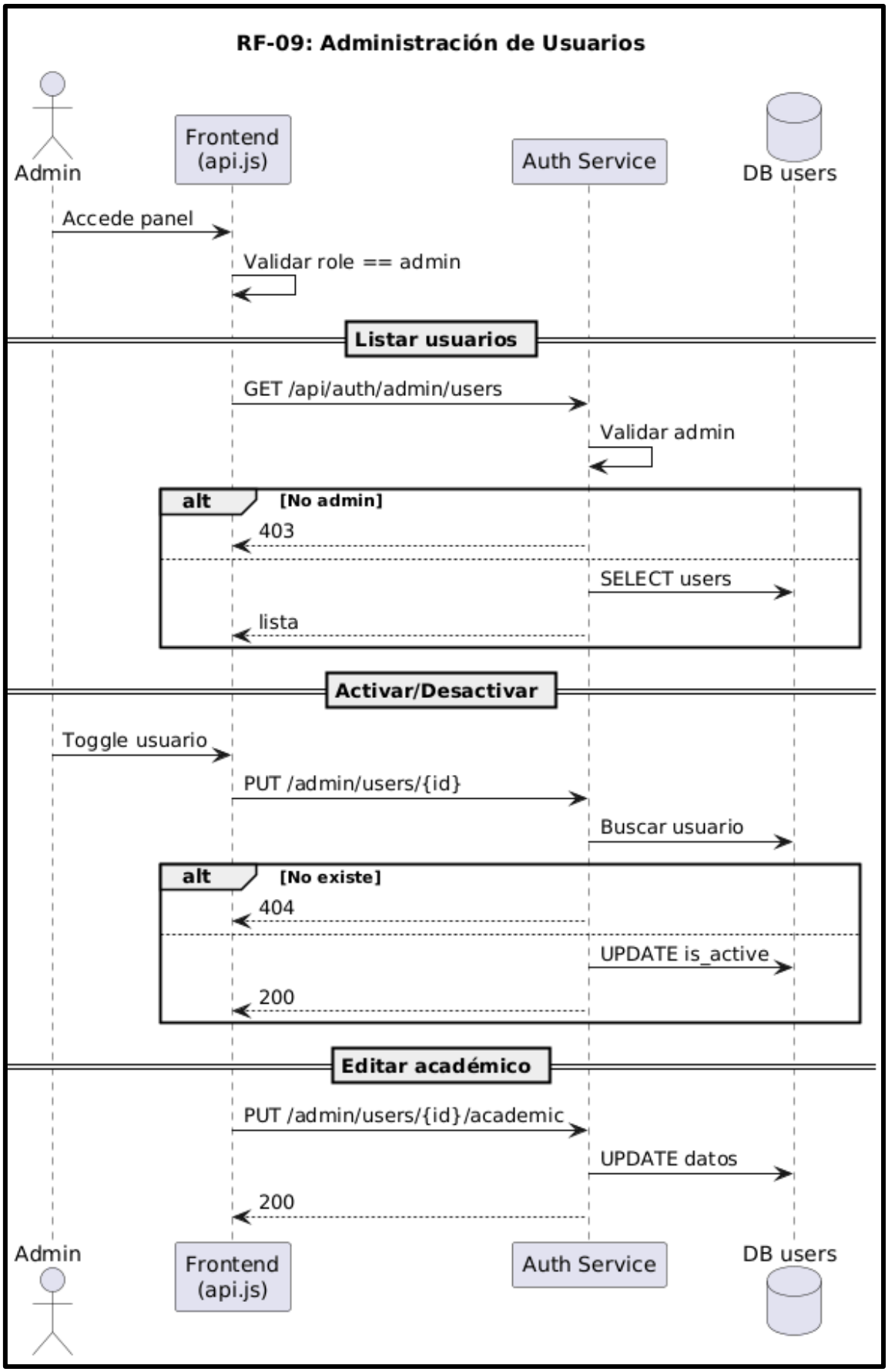
Se describe el proceso de comunicación entre usuarios mediante mensajes privados. El sistema valida la relación entre usuarios, almacena los mensajes y permite su recuperación en tiempo casi real.

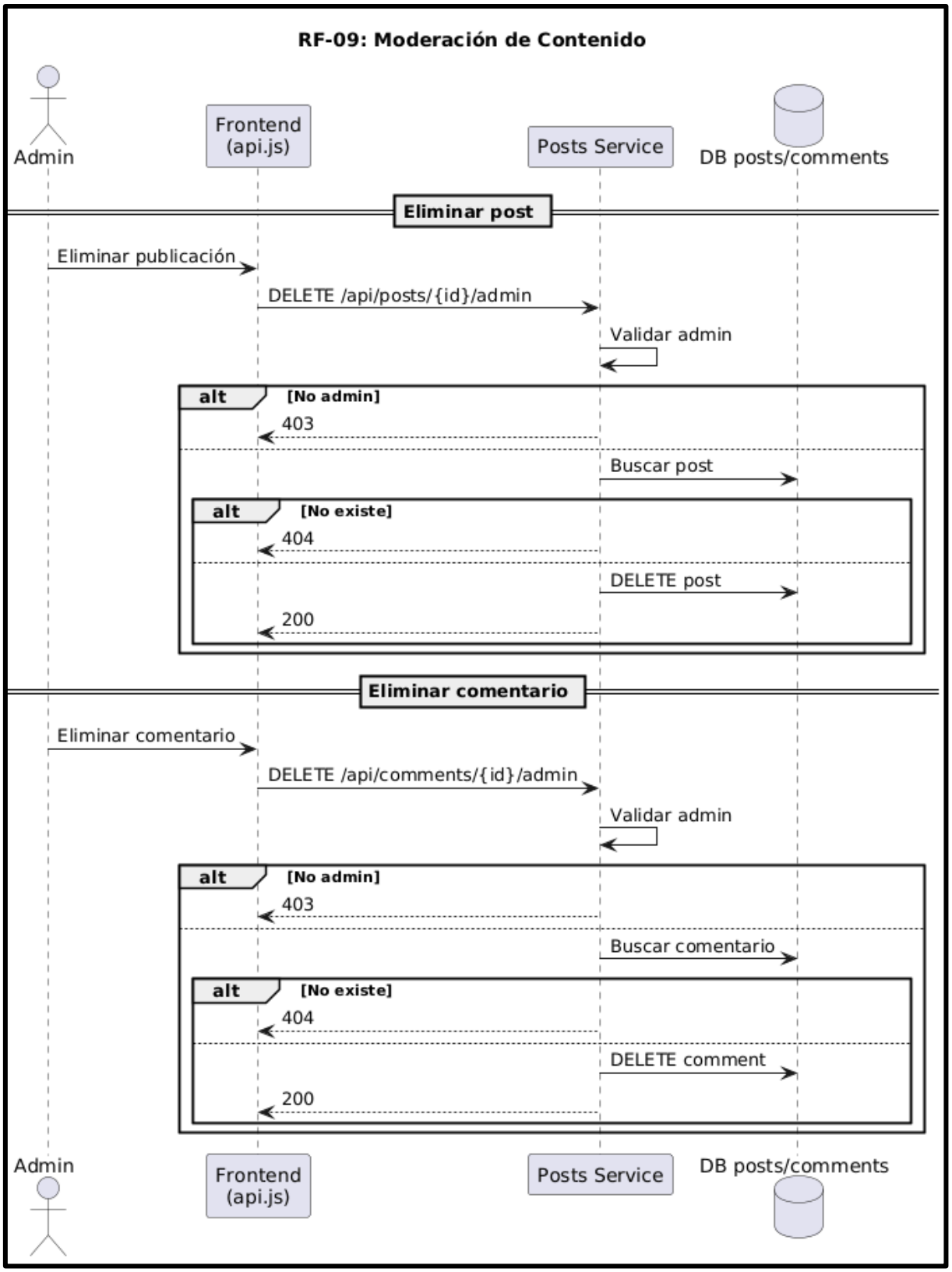




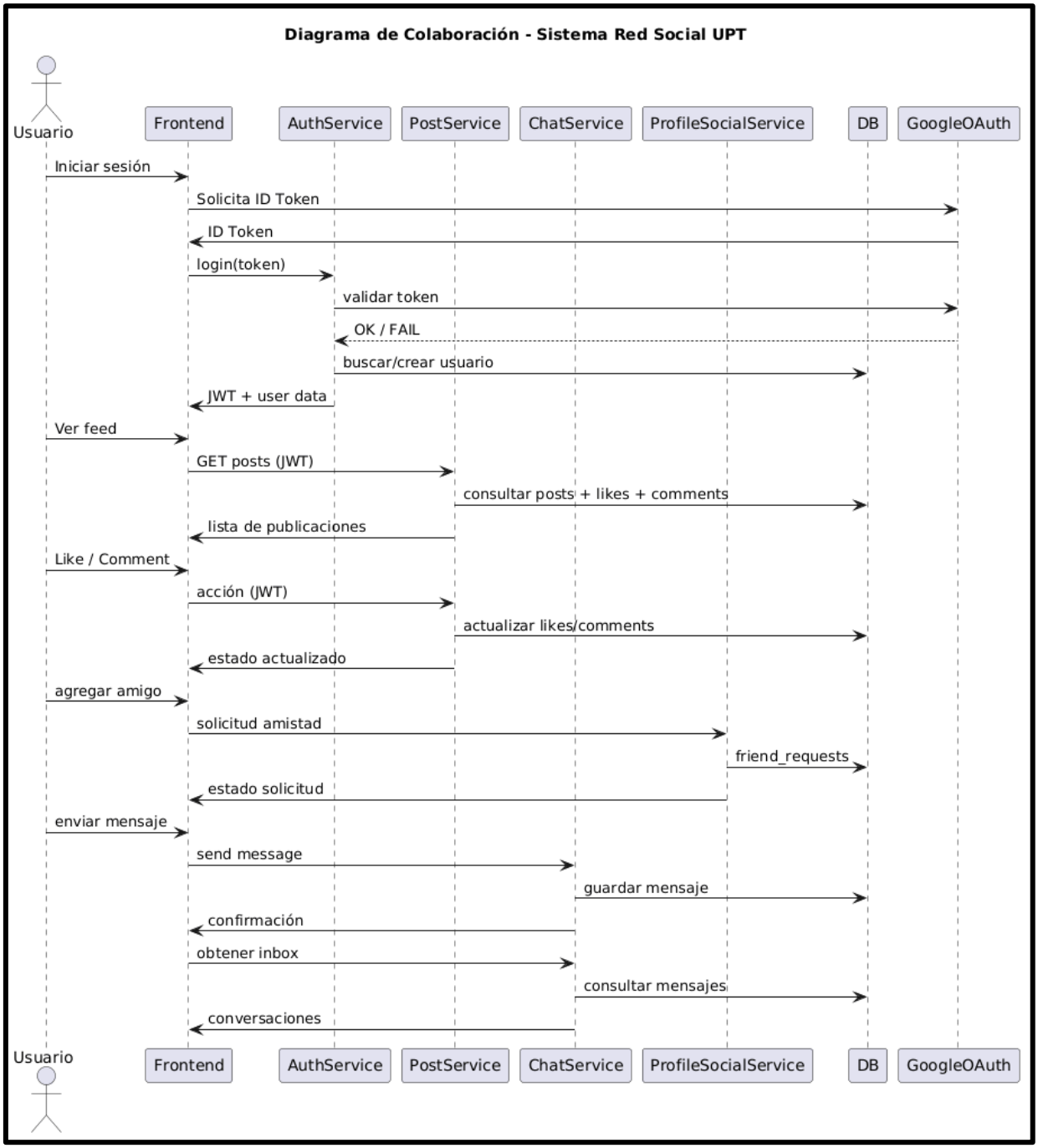
RF-09: Panel de Administración

Se representa el flujo de acceso y gestión del panel administrativo. El sistema valida el rol del usuario y permite la administración de usuarios y contenido, incluyendo la edición, activación/desactivación y eliminación de publicaciones o comentarios.

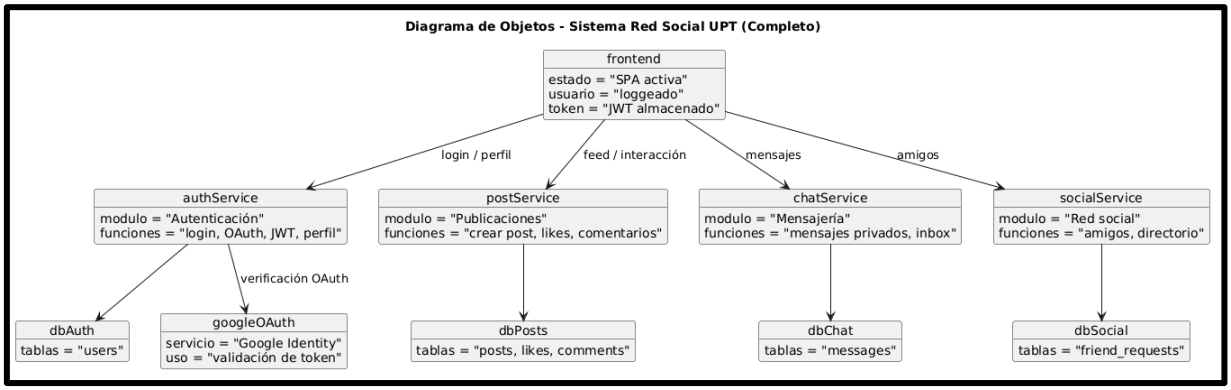

RF-09: Administración de Usuarios
Admin
Frontend (api.js)
Auth Service
DB users
Accede panel
Validar role == admin
Listar usuarios
GET /api/auth/admin/users
Validar admin
alt
[No admin]
403
SELECT users
lista
Activar/Desactivar
Toggle usuario
PUT /admin/users/{id}
Buscar usuario
alt
[No existe]
404
UPDATE is_active
200
Editar académico
PUT /admin/users/{id}/academic
UPDATE datos
200
Admin
Frontend (api.js)
Auth Service
DB users


RF-09: Moderación de Contenido
Admin
Frontend (api.js)
Posts Service
DB posts/comments
Eliminar post
Eliminar publicación
DELETE /api/posts/{id}/admin
Validar admin
alt
[No admin]
403
Buscar post
alt
[No existe]
404
DELETE post
200
Eliminar comentario
Eliminar comentario
DELETE /api/comments/{id}/admin
Validar admin
alt
[No admin]
403
Buscar comentario
alt
[No existe]
404
DELETE comment
200
Admin
Frontend (api.js)
Posts Service
DB posts/comments

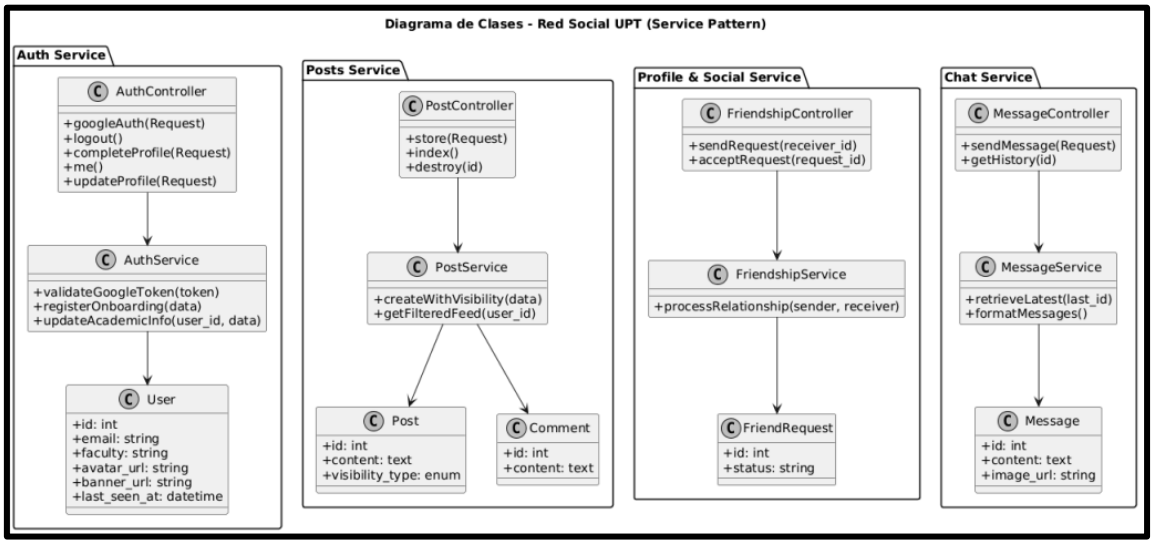
3.2.3. Diagrama de Colaboración (vista de diseño)



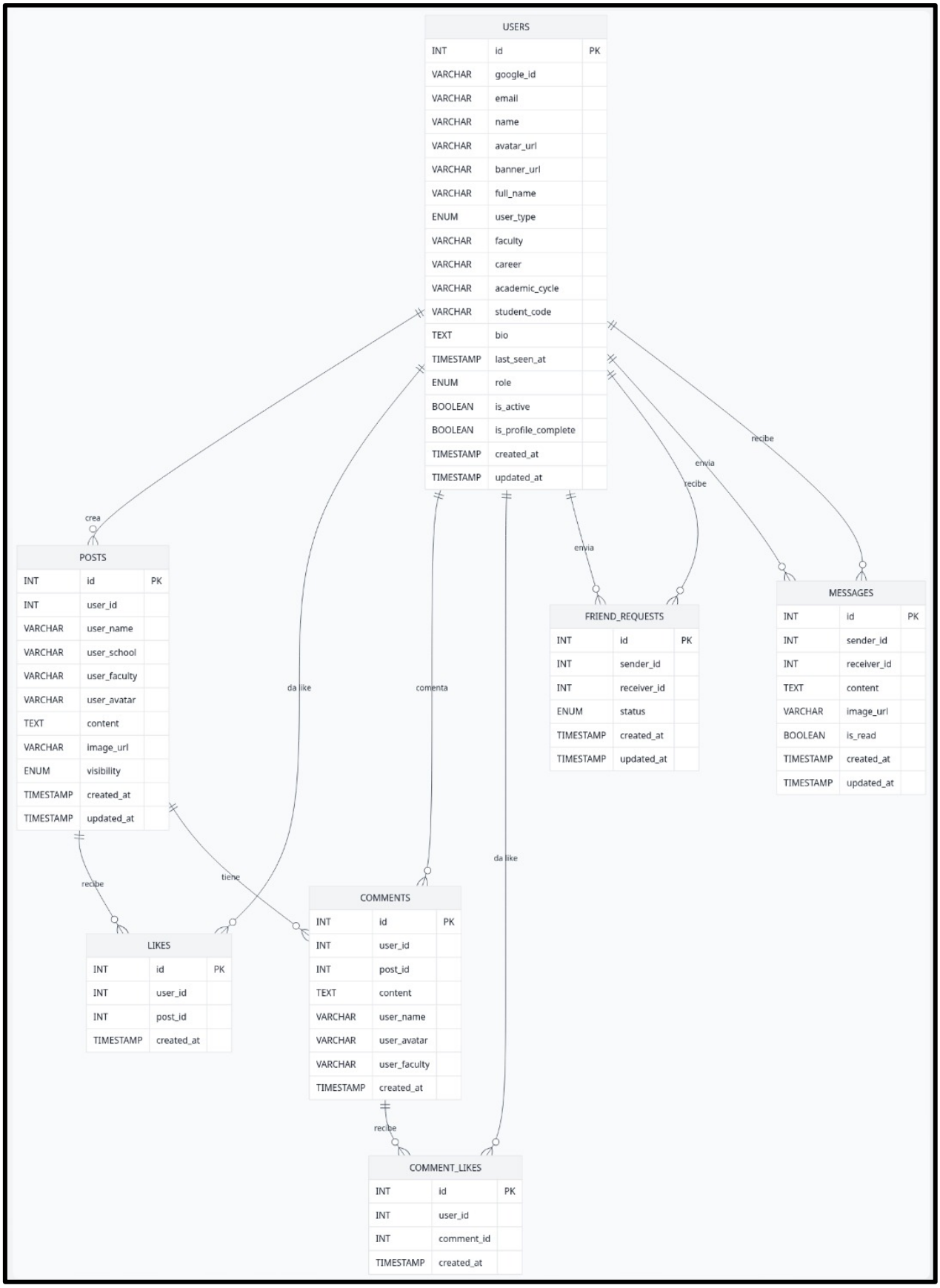
3.2.4. Diagrama de Objetos



3.2.5. Diagrama de Clases

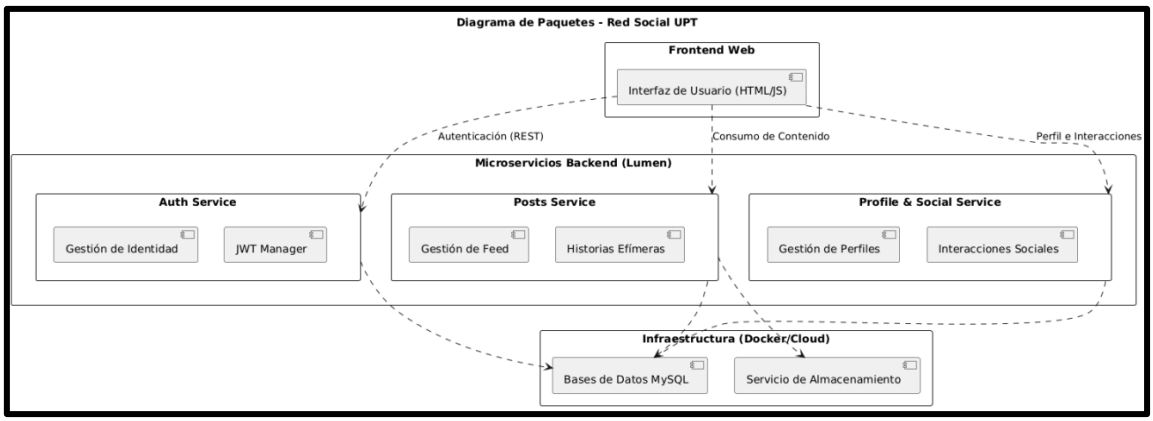


3.2.6. Diagrama de Base de datos (relacional o no relacional)

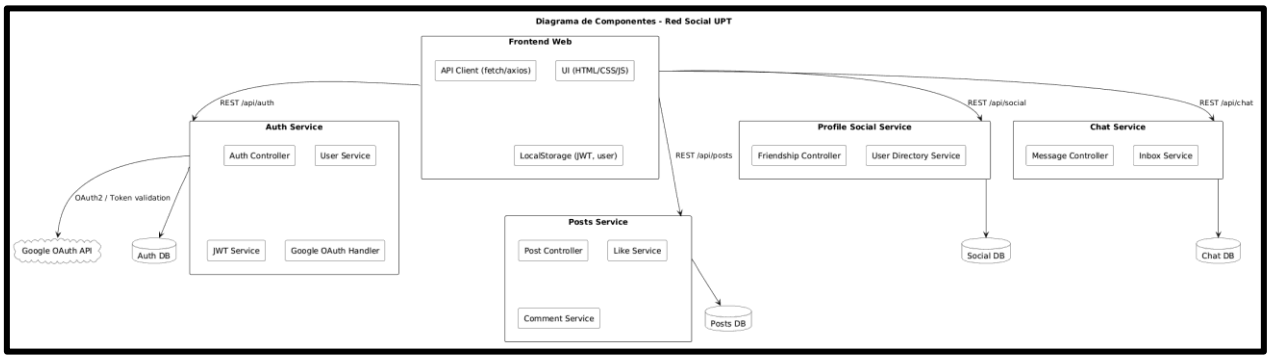


3.3. Vista de Implementación (vista de desarrollo)

3.3.1. Diagrama de arquitectura software (paquetes)

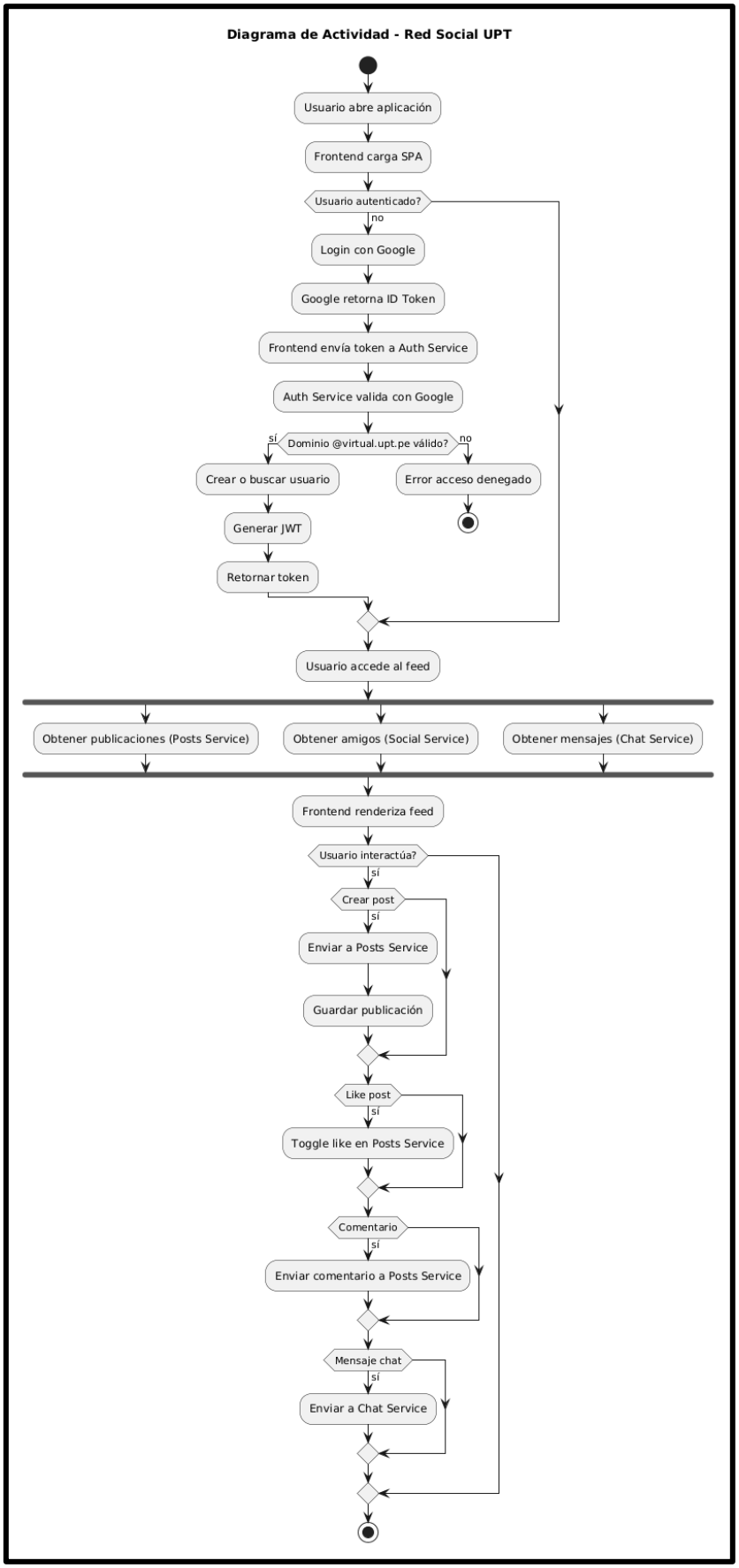


3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)



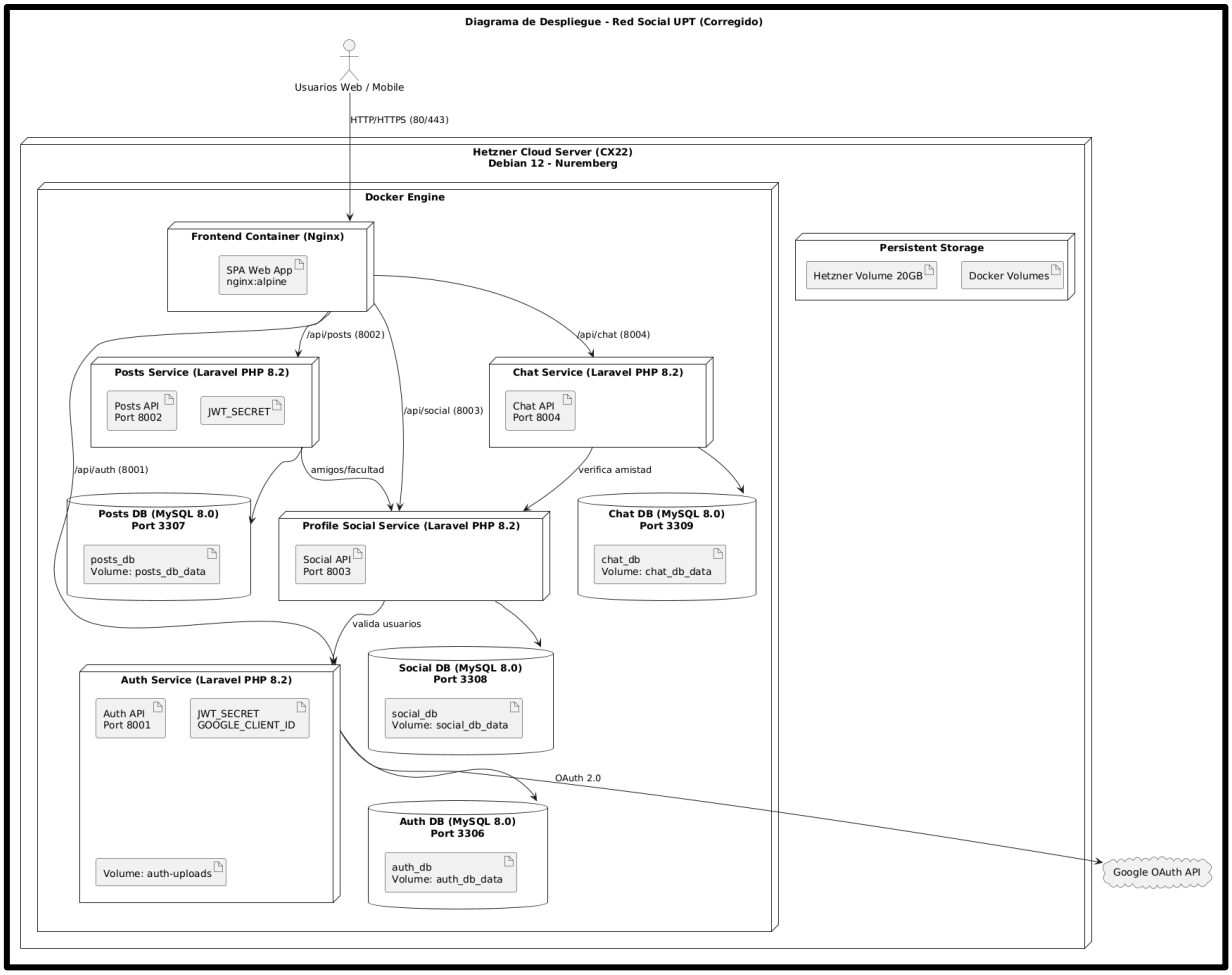
3.4. Vista de procesos

3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)



3.5. Vista de Despliegue (vista física)

3.5.1. Diagrama de despliegue



4. ATRIBUTOS DE CALIDAD DEL SOFTWARE

Los atributos de calidad (QAs) son propiedades medibles y evaluables de un sistema que permiten determinar el grado en que este satisface las necesidades de los stakeholders. A diferencia de los requerimientos funcionales, los atributos de calidad se centran en cómo el sistema realiza sus funciones, considerando aspectos como seguridad, rendimiento, usabilidad y mantenibilidad.

En el sistema **Red Social UPT**, los atributos de calidad son fundamentales debido a la naturaleza del sistema, el cual maneja información de usuarios, comunicación en tiempo real y acceso restringido mediante autenticación institucional.

Escenario de Funcionalidad

El sistema Red Social UPT proporciona un conjunto de funcionalidades orientadas a la interacción social dentro de la comunidad universitaria, tales como autenticación, publicaciones, comentarios, reacciones, gestión de perfiles y mensajería.

Escenario:

- Fuente: Usuario (estudiante o docente)
- Estímulo: El usuario realiza una acción (crear publicación, comentar, dar like)
- Entorno: Sistema en operación normal
- Respuesta: El sistema procesa la solicitud correctamente y refleja los cambios en la interfaz
- Medida de respuesta: La operación se completa sin errores y con consistencia de datos

Este atributo asegura que el sistema cumple con los requerimientos funcionales definidos.

Escenario de Usabilidad

La usabilidad del sistema se enfoca en la facilidad de aprendizaje, eficiencia de uso y satisfacción del usuario al interactuar con la plataforma.

Escenario:

- **Fuente:** Usuario nuevo
- **Estímulo:** Accede por primera vez al sistema
- **Entorno:** Navegador web en dispositivo estándar
- **Respuesta:** El usuario puede registrarse, completar su perfil y navegar por el sistema sin dificultad
- **Medida de respuesta:** El usuario logra completar acciones básicas sin asistencia externa

El sistema utiliza una interfaz web responsiva y simple, facilitando la navegación y reduciendo la curva de aprendizaje.

Escenario de confiabilidad

La confiabilidad del sistema se relaciona con la capacidad de operar correctamente y de manera segura, protegiendo la información y garantizando su disponibilidad.

Escenario:

- **Fuente:** Sistema o usuario
- **Estímulo:** Intento de acceso o interacción con datos
- **Entorno:** Operación normal o intento de acceso no autorizado
- **Respuesta:** El sistema valida la autenticación mediante Google OAuth y JWT, permitiendo o denegando el acceso
- **Medida de respuesta:** No se permite acceso a usuarios no autorizados y los datos permanecen íntegros

Se implementan mecanismos de seguridad como autenticación externa, validación de dominio institucional y comunicación segura entre microservicios.

Escenario de rendimiento

El rendimiento mide la eficiencia del sistema en términos de tiempo de respuesta y uso de recursos.

Escenario:

- **Fuente:** Usuario
- **Estímulo:** Solicitud de carga del feed o envío de mensaje
- **Entorno:** Sistema con múltiples usuarios concurrentes
- **Respuesta:** El sistema responde mostrando el contenido solicitado
- **Medida de respuesta:** Tiempo de respuesta menor a 2 segundos en operaciones principales

El uso de arquitectura de microservicios permite distribuir la carga y mejorar el rendimiento general del sistema.

Escenario de mantenibilidad

La mantenibilidad se refiere a la facilidad con la que el sistema puede ser modificado, corregido o ampliado.

Escenario:

- **Fuente:** Desarrollador
- **Estímulo:** Necesidad de agregar una nueva funcionalidad
- **Entorno:** Sistema en desarrollo o mantenimiento
- **Respuesta:** Se modifica un microservicio sin afectar los demás
- **Medida de respuesta:** Cambios implementados con bajo impacto en otros componentes

El uso de microservicios, Docker y separación por módulos permite una alta mantenibilidad del sistema.

Otros Escenarios

Escenario de Seguridad

El sistema maneja información sensible, por lo que la seguridad es un atributo crítico.

Escenario:

- **Fuente:** Usuario externo o atacante
- **Estímulo:** Intento de acceso sin credenciales válidas
- **Entorno:** Sistema expuesto en internet
- **Respuesta:** El sistema bloquea el acceso y no permite interacción
- **Medida de respuesta:** 100% de accesos no autorizados rechazados

Se utilizan mecanismos como:

- Google OAuth
- Tokens JWT
- Validación de dominio institucional

Escenario de Disponibilidad

El sistema debe estar disponible para los usuarios en todo momento.

Escenario:

- **Fuente:** Usuario
- **Estímulo:** Acceso al sistema
- **Entorno:** Sistema desplegado en VPS
- **Respuesta:** El sistema responde correctamente
- **Medida de respuesta:** Alta disponibilidad del servicio

El uso de contenedores Docker facilita la recuperación ante fallos.